DeepSeek CSA 逐模块公式讲义

第一阶段：前向与手写反向　第二阶段：片上内存受限的融合 kernel

AISys-course | 2026 年 10 月

1 讲义怎么读

本讲义带大家从头实现 DeepSeek-V4 的核心注意力模块——Compressed Sparse Attention（CSA，压缩稀疏注意力）。我们会逐一拆解层内的每个子模块，讲清楚它为什么这么设计、前向怎么算、反向梯度怎么推。全篇最重要的约定只有一条：**公式里用打字机字体写的名字，直接对应代码里的变量名或属性名**（如 `qr`、`topk_idxs`）。模块的参数与子模块定义在 `handout/csa/modules.py` 中；你需要完成的代码分两阶段，第一阶段填在 `handout/csa/attention.py`，第二阶段填在 `handout/csa/fused.py`。

后面每一节都按“代码接口 → 前向推导 → 边界情况 → 反向传播”的顺序展开。前向部分给出完整公式；反向部分则说明输入输出形状、需要累加的参数梯度以及容易踩坑的边界情况，并附上推导提示。建议先按第 2.1 节的方法在草稿纸上推出反向式子再编码，并用 `handout/csa/gradcheck.py` 做有限差分检验（做法见附录 B）。

下表列出所有需要完成的模块。CSA 的一句话思路（第 3 节展开）：普通注意力让每个 query 读前面所有 token 的 KV；CSA 只让它读两部分——最近一段窗口内每个 token 的 KV，以及从更早的历史里挑出的少量压缩条目（compressed entry，每 4 个 token 压成一条）。

需要实现的函数 / 类	章节	作用
第一阶段（`attention.py`）：用普通 NumPy 写出各模块的前向和反向		
`Linear.backward`	4	线性层 $y = xW^\top$ 的反向（前向由框架提供）
`rms_forward`、`rms_backward`、`RMSNorm`	5	RMS 归一化：把每个向量除以它的均方根；`RMSNorm` 再乘一个可学习的逐通道权重
`Compressor.forward/backward`	7	Compressor（压缩器）：把序列每 4 个 token 压成一条压缩条目
`Indexer.forward/backward_scores`	8	Indexer（索引器）：为每个 query 挑出最相关的几条压缩条目（不能挑未来的），以及打分的反向
`sparse_attn`、`sparse_attn_backward`	9	注意力计算本身：每个 query 只读窗口内的 KV 和挑出的压缩条目
`Attention.forward/backward`	10	把以上模块串成完整的一层，并写出整层反向
第二阶段（`fused.py`）：在 48 KiB 片上内存的限制下，把同样的计算改写成分块 kernel		
`fused.sparse_attn`	13	第 9 节注意力的前向：条目分批读入片上，边读边累加（在线 softmax）

需要实现的函数 / 类	章节	作用
`fused.sparse_attn_backward`	14	第 9 节注意力的反向：不保存注意力概率，在片上重新算出来
`fused.gated_pool(_backward)`	15	第 7 节 Compressor 里按门控权重求加权平均的一步（含反向）：中间结果只留在片上
`fused.index_topk`	16	第 8 节的打分和挑选：不把完整的分数矩阵写回显存，边算边保留最好的几条
`fused.attention_forward`	17	用以上 kernel 和框架提供的矩阵乘等，在模拟器上跑通第 10 节的整层前向

第 6 节介绍的 RoPE 和 Hadamard 变换由框架直接提供，但写反向时需要用到它们的逆变换（转置）。文末附录 A 整理了一份提交前的自查清单，附录 B 介绍怎样用有限差分检查手写反向；附录 C 简要补充了作业略去的 YaRN 频率、decode 缓存、低比特量化和张量并行；附录 D 则按名称列出了每个模块在 DeepSeek-V4 官方源码中对应的类和函数。

1.1 怎么对照架构图

图 1 取自开源项目 CalvinXKY/InfraTech 绘制的 DeepSeek-V4 Pro 全景架构图。本作业聚焦其中高亮的四个部分，后续章节会把对应局部放大，和公式放在一起对照：橙框是完整的 CSA 注意力层（第 3、9、10 节），绿框是负责选块的 Indexer（第 8 节），蓝框和紫框分别是 Compressor 及其跨块重叠排布（第 7 节）。看图时留意三处和作业代码的差异：

- **图画的是逐 token decode，作业写的是整段 prefill**。因此图里代表当前 token 数的维度写成了 `1`，代表历史压缩块数的维度写成了 T 或 t；在我们的 prefill 实现里，它们分别对应序列长度 S 和完整压缩块数 N。
- **图上标的是 V4-Pro 生产模型的真实超参**（如 `hidden_size=7168`、`head_dim=512`、`heads=128`）；作业测试使用第 2 节的缩小版配置，以便在 CPU 上几秒内跑完梯度检查。
- **图中的 Quant 框在作业里是恒等映射**。真实推理在激活上做 FP8 / FP4 量化，作业保留了 `act_quant` 和 `fp4_act_quant` 的调用位置，但它们原样返回输入。

2 记号与约定

- **下标从 0 开始：** b 表示 batch，t 表示 token 在序列中的位置（prefill 一次喂入整条序列，t 即绝对位置），h 表示注意力头，c 表示通道，j 表示压缩块编号，r 表示槽位（slot）编号。
- **行向量约定：** 特征维都在最后一维。线性层权重存为 `weight[out, in]`，因此前向计算写作 $x\,W^{\top}$。
- **代码同名：** 打字机字体如 `qr`、`alpha`、`topk_idxs` 与代码变量名严格一致；张量形状记作 $[B, S, H, D]$，对应 NumPy 的 `.shape`。
- **梯度记号：** $\mathrm{d}X$ 表示标量损失 L 对张量 X 的偏导 $\partial L/\partial X$，形状与 X 相同，在代码里写作 `dX`（例如 dq 对应 `dq`）。

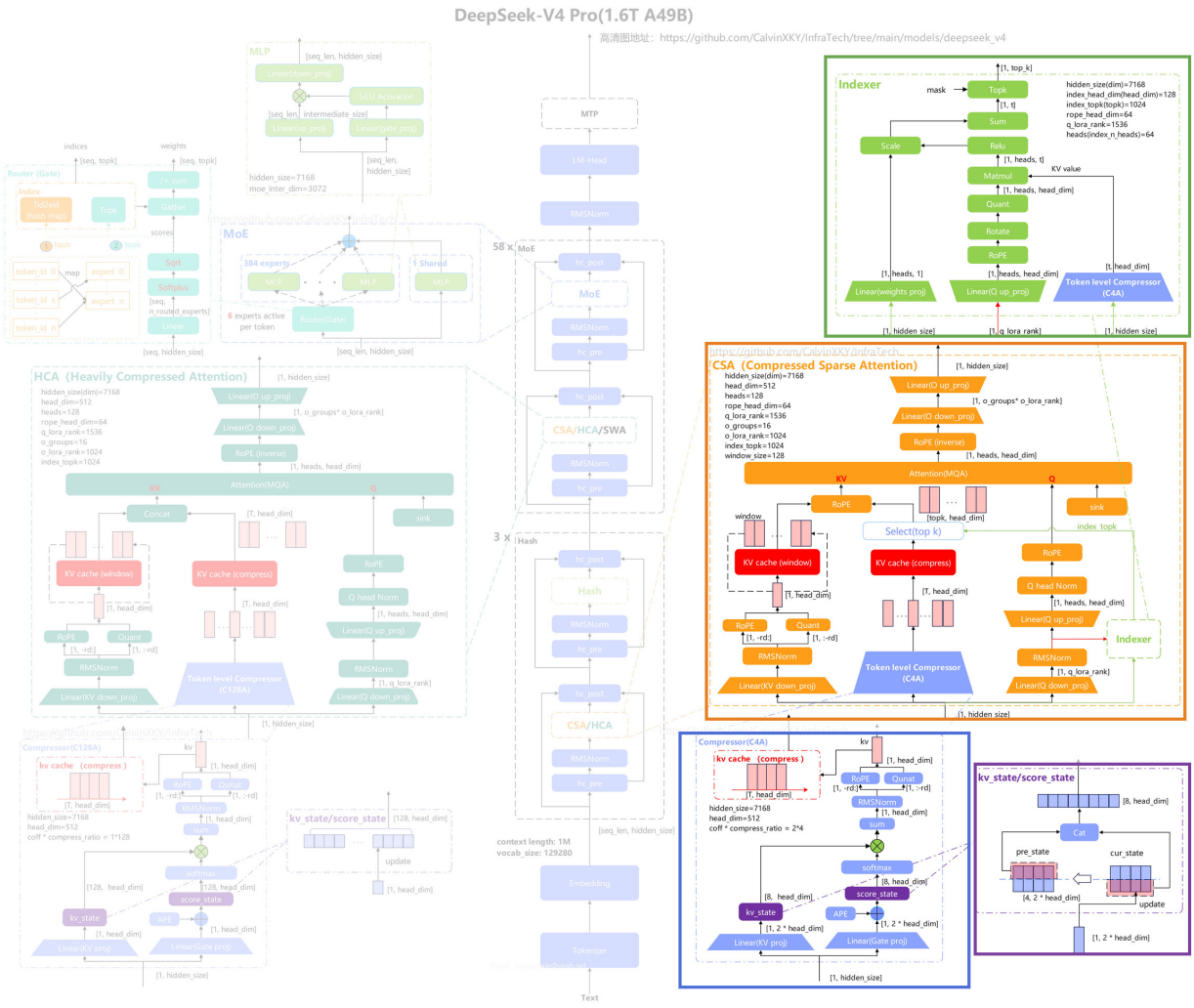


图 1 DeepSeek-V4 Pro 架构全景。高亮的四块对应本作业实现的 CSA 层；其余模块（HCA、MoE、Hash 层、层间超连接 hc_pre / hc_post 等）不在作业范围内。

- **参数梯度一律累加（+=）：** 与 PyTorch 一致，调用方会在反向前先调 `zero_grad()`，各层的 `backward` 必须把算出的参数梯度**加**进 `self._grads`，不能直接覆盖。
- **常用算符：** $\odot$ 表示逐元素相乘；mean_c 表示沿最后一维求均值；$\mathbb{I}[\cdot]$ 为指示函数（条件成立取 1，否则取 0）；被掩码屏蔽的 logit 记为 $-\infty$，经 exp 后自然变成 0。

符号	配置字段 / 属性	含义
B, S	—	batch 大小、序列长度
E	`dim`	输入输出的维度
H, D	`n_heads`、`head_dim`	attention query 的头数、每头维度
R	`q_lora_rank`	共享低秩 query 的维度
d_r	`rope_head_dim`	每个向量末尾参与 RoPE 的通道数（偶数）
G, R_o	`o_groups`、`o_lora_rank`	输出分组数、每组的低秩维度
w	`window_size`	局部窗口宽度，包含当前 token
m	`compress_ratio` $= 4$	每 m 个 token 一个压缩块

符号	配置字段 / 属性	含义
N	$\lfloor S/m \rfloor$	完整压缩块的数量
H_I, D_I	`index_n_heads`、`index_head_dim`	Indexer 的头数、每头维度（D_I 为 2 的幂）
k	`index_topk`	每个 query 最多选的压缩块数
ε	`norm_eps`	RMS 归一化中的 ε

默认配置 `ModelArgs()` 为 $E = 12, H = 4, D = 8, R = 6, d_r = 4, G = 2, R_o = 4, w = 128, H_I = 2, D_I = 8, k = 2$，只用于快速测试。

2.1 怎样推反向：按分量写链式法则

每个 `backward` 做的事都一样：已知标量损失 L 对输出 Y 的梯度 $\mathrm{d}Y$（与 Y 同形状），求 L 对输入 X 的梯度 $\mathrm{d}X$。X 的每个元素 X_a 通过所有用到它的输出元素 Y_k 影响 L，按链式法则，

$$\mathrm{d}X_a = \sum_k \mathrm{d}Y_k \, \frac{\partial Y_k}{\partial X_a}.$$

把所有 $\partial Y_k / \partial X_a$ 排成一个矩阵，就是雅可比矩阵 J；上式写成矩阵形式是 $\mathrm{d}X = J^\top \mathrm{d}Y$，称为向量-雅可比积（vector-Jacobian product，VJP），PyTorch 的 autograd 算的也是它。不过 J 从来不需要真的构造出来：它的元素个数是 Y 与 X 元素个数的乘积。比如输入为 $[4096, 64]$ 的 $64 \to 64$ 线性层，输出对权重的雅可比矩阵约有 10^9 个元素，其中 98% 以上是 0。推导时直接在分量上计算上面的求和，分三步：

1. **写出分量式**。把前向写成带下标的标量等式，例如线性层的 $y_{n,o} = \sum_i x_{n,i} W_{o,i}$。
2. **对所有依赖项求和**。对要求梯度的那个元素，找出所有用到它的输出元素，把“上游梯度 × 局部偏导”全部加起来；与它无关的输出贡献为 0，直接略去。
3. **写回数组运算并检查形状**。把下标求和改写成矩阵乘、广播求和或 `np.einsum`，并确认结果与被求导的数组形状相同。

本作业的反向里反复用到下面两条规律，先各用一个最简单的例子说明。

规律一：参数被所有位置共用时，梯度要对所有位置求和。以作业里没有的偏置为例：$y_{n,o} = z_{n,o} + b_o$，其中 n 是把 batch、序列位置等前导维度展平后的行号。偏置 b_c 被每一行的第 c 个输出 $y_{n,c}$ 各用了一次；b_c 增加 1，这些输出都正好增加 1，其余输出不变。所以 L 对 b_c 的梯度就是这些输出的上游梯度之和：

$$\mathrm{d}b_c = \sum_n \mathrm{d}y_{n,c}.$$

比如一共 3 行，就是 $\mathrm{d}y_{0,c} + \mathrm{d}y_{1,c} + \mathrm{d}y_{2,c}$。`Linear` 的 W、`RMSNorm` 的 `weight`、Compressor 的 `ape` 都是这种被许多位置共用的参数，它们的梯度都要把所有用到它们的位置的贡献加起来。

规律二：一个量流向多处时，各处传回的梯度相加。设 x 同时用来算 $u = 3x$ 和 $v = x^2$，损失 L 又依赖 u 和 v。x 一变，会同时通过 u 和 v 影响 L，两条路径的贡献要加起来：

$$\mathrm{d}x = \mathrm{d}u \cdot \frac{\partial u}{\partial x} + \mathrm{d}v \cdot \frac{\partial v}{\partial x} = 3\,\mathrm{d}u + 2x\,\mathrm{d}v.$$

本作业里会遇到的例子：`qr` 同时流向 attention query 和 Indexer 两条支路（第 10 节）；一个 KV 条目既当 key 又当 value，还会被多个头、多个 query 读到（第 9 节）；RMS 归一化里 `inv_rms` 由整行算出，所以每个 x_c 除了直接影响 y_c，还会通过 `inv_rms` 影响整行的输出（第 5 节）。

例：softmax 的反向。这个例子把前面的三步完整走一遍。

1. **写出分量式：** $p_j = e^{s_j}/Z$，其中分母 $Z = \sum_u e^{s_u}$。
2. **求局部偏导：** s_i 一变，所有 p_j 都会变，因为分母 Z 里有 e^{s_i}；只有 $j = i$ 时分子 e^{s_j} 也会变。按商的求导法则：
$$\frac{\partial p_j}{\partial s_i} = p_j\,\mathbb{I}[i=j] - p_j\,p_i,$$
其中 $\mathbb{I}[i=j]$ 是指示函数：$i=j$ 时为 1，否则为 0。
3. **对所有依赖项求和：** 每个 p_j 都依赖 s_i，所以要对 j 求和，并把两项分开算：
$$\begin{aligned}\mathrm{d}s_i &= \sum_j \mathrm{d}p_j\,\frac{\partial p_j}{\partial s_i} = \sum_j \mathrm{d}p_j\,p_j\,\mathbb{I}[i=j] \;-\; p_i \sum_j p_j\,\mathrm{d}p_j \\ &= p_i\,\mathrm{d}p_i - p_i\sum_j p_j\,\mathrm{d}p_j = p_i\Big(\mathrm{d}p_i - \sum_j p_j\,\mathrm{d}p_j\Big).\end{aligned}$$
第一项里的指示函数只留下 $j=i$ 那一项，所以得到 $p_i\,\mathrm{d}p_i$。

写成 NumPy 只需一行：先算出一个标量 $\sum_j p_j\,\mathrm{d}p_j$，再逐元素运算。由 $\partial p_j/\partial s_i$ 排成的雅可比矩阵只在推导里出现，代码里从来不用构造它。

```
ds = p * (dp - (p * dp).sum(axis=-1, keepdims=True))
```

第 9 节带 sink 的 softmax 反向会用到这个结果。推出任何反向公式之后，都可以用附录 B 的有限差分检查一遍。

3 一次前向的完整路线

标准自注意力让每个 query 去读前面所有 token 的 KV；序列一长，KV 缓存和计算量都扛不住。CSA 的做法是把上下文拆成“近处细看、远处粗看”：窗口内最近的 w 个 token 保留逐 token 的局部 KV；更早的历史则按每 $m=4$ 个 token 一块压成粗粒度条目，由轻量的 Indexer 挑出最相关的 k 个块。这样每个 query 最多只需要读 $w+k$ 个 KV 条目。

反映到代码里，`Attention.forward(x)` 接收 $x \in \mathbb{R}^{B\times S\times E}$，输出同形状的 y。整层前向沿着三条支路推进，最后在稀疏注意力处汇合（图 2）：

1. **Query 支路。** x 先做一次低秩下投影得到共享表示 `qr`，一方面向上投影成 H 个头的 attention query `q`，另一方面直接喂给 Indexer 复用（Indexer 由此算出自己的 query，下文称 index query，以示区别）。
2. **KV 支路。** 一边为每个 token 算一个局部 KV 向量 `local`，另一边用 Attention 自己的 Compressor（`self.compressor`）每 m 个 token 压出一个粗粒度条目 `compressed`。两段拼在一起组成共享的 `kv` 表（前 S 行是局部条目，后 N 行是压缩条目，所以压缩块 j 在 `kv` 里的行号是 $S+j$）。CSA

采用 MQA（multi-query attention），所有注意力头共用这张 `kv` 表，且每个条目同时充当 key 和 value。

3. **路由支路（构造 `topk_idxs`）**。每个 query 要读哪些行，由两部分拼成：前半是滑动窗口给出的最近 $\min(S, w)$ 个局部位置，后半是 Indexer 打分挑出的至多 k 个压缩块行号。

拿到 `q`、`kv` 和路由表 `topk_idxs` 后，`sparse_attn` 只收集被选中的条目做注意力加权；出来的结果再做一次逆 RoPE，经过分组输出投影 `wo_a` 和 `wo_b`，就得到整层输出 y。

需要注意，Attention 层和 Indexer 各自配了一个结构相同但参数独立的 Compressor：Attention 的 Compressor（`self.compressor`）压出的条目真正写进 `kv` 参与注意力加权；Indexer 的 Compressor（`self.indexer.compressor`）只产生打分用的低维 key，用来决定选哪些块，梯度也不会跨过 top-k 传回主损失。

图 2 中的模块	对应代码	章节
Linear(Q down_proj) → RMSNorm	`wq_a` → `q_norm`，输出 `qr`	10
Linear(Q up_proj) → Q head Norm → RoPE	`wq_b` → 每头 `rms_forward` → `partial_rope`	10
Linear(KV down_proj) → RMSNorm → RoPE / Quant	`wkv` → `kv_norm` → `partial_rope` / `act_quant`	10
KV cache (window)	`kv` 的前 S 行（`local`）及滑动窗口槽位	10
Token level Compressor (C4A) → KV cache (compress)	`self.compressor`，写入 `kv` 的后 N 行	7
Indexer → Select(top k)	`self.indexer`，生成 `topk_idxs` 的压缩槽位	8
sink	`attn_sink`	9
Attention(MQA)	`sparse_attn`	9
RoPE (inverse) → Linear(O down_proj) → Linear(O up_proj)	逆 `partial_rope` → 分组 `wo_a` → `wo_b`	10

张量	形状	说明
`qr`	$[B, S, R]$	attention query 与 Indexer 共享的低秩表示 `q_norm(wq_a(x))`
`q`	$[B, S, H, D]$	attention query（经 `wq_b`、每头无权重 RMS、RoPE）
`local`	$[B, S, D]$	逐 token 局部 KV（经 `wkv`、`kv_norm`、RoPE）
`compressed`	$[B, N, D]$	Attention 的 Compressor `self.compressor(x)` 输出的压缩 KV
`kv`	$[B, S+N, D]$	拼接后的完整 KV 表：前 S 行 `local`，后 N 行 `compressed`
`topk_idxs`	$[B, S, \min(S, w) + \min(k, N)]$	每个 query 要读取的 `kv` 行号（窗口槽位在前，压缩槽位在后；-1 为空）
o	$[B, S, H, D]$	`sparse_attn` 的输出

张量	形状	说明
`grouped`	$[B, S, G, HD/G]$	o 做逆 RoPE 后按 G 个输出组重排
`reduced`	$[B, S, G, R_o]$	`grouped` 与 `wo_a` 做分组投影后的低秩结果
y	$[B, S, E]$	经 `wo_b` 上投影还原的层输出

4 线性层

代码： 框架提供 `LinearBase.forward`；需完成 `Linear.backward(dy)`。

4.1 前向（框架提供）

线性层没有偏置，权重矩阵为 $W = \texttt{weight} \in \mathbb{R}^{\text{out}\times\text{in}}$。输入 x 的最后一维长度为 in，前面可以带任意多个前导维度（如 $[B, S, \text{in}]$）。按分量和按矩阵分别写作：

$$y_{\ldots,o} = \sum_i x_{\ldots,i} W_{o,i},$$
$$y = x\, W^\top.$$

框架的 `LinearBase.forward` 在算出 y 的同时，会把输入直接保存在 `self._x` 上。这也是为什么规则要求同一个 `Linear` 实例在一次前向里最多调用一次——调第二次就会把前一次存的 `self._x` 冲掉，反向就乱套了。

4.2 反向

`backward(dy)` 接收与 y 同形状的上游梯度 $\mathrm{d}y$，返回与 x 同形状的 $\mathrm{d}x$，并把权重梯度 $\mathrm{d}W \in \mathbb{R}^{\text{out}\times\text{in}}$ 累加进 `self._grads["weight"]`。

提示

按第 2.1 节的三步推：把前导维度展平成行号 n 后，$y_{n,o}$ 对 $W_{o,i}$ 的偏导就是同一行的 $x_{n,i}$；W 被所有行共用，所以 $\mathrm{d}W$ 要把每一行的贡献加起来，每行贡献的是一个外积。$\mathrm{d}x$ 同理，最后检查两者的形状分别是 $[\ldots, \text{in}]$ 和 $[\text{out}, \text{in}]$。

5 RMS 归一化与 RMSNorm

代码： 需完成 `rms_forward(x, eps)`、`rms_backward(dy, cache)`、`RMSNorm.forward/backward`。

CSA 里用到了两种 RMS 归一化：attention query 在分出 H 个头之后，对每个头单独做一次**不带可学习权重**的 RMS 归一化（Q head Norm）；而 `q_norm`、`kv_norm` 以及两个 Compressor 里的

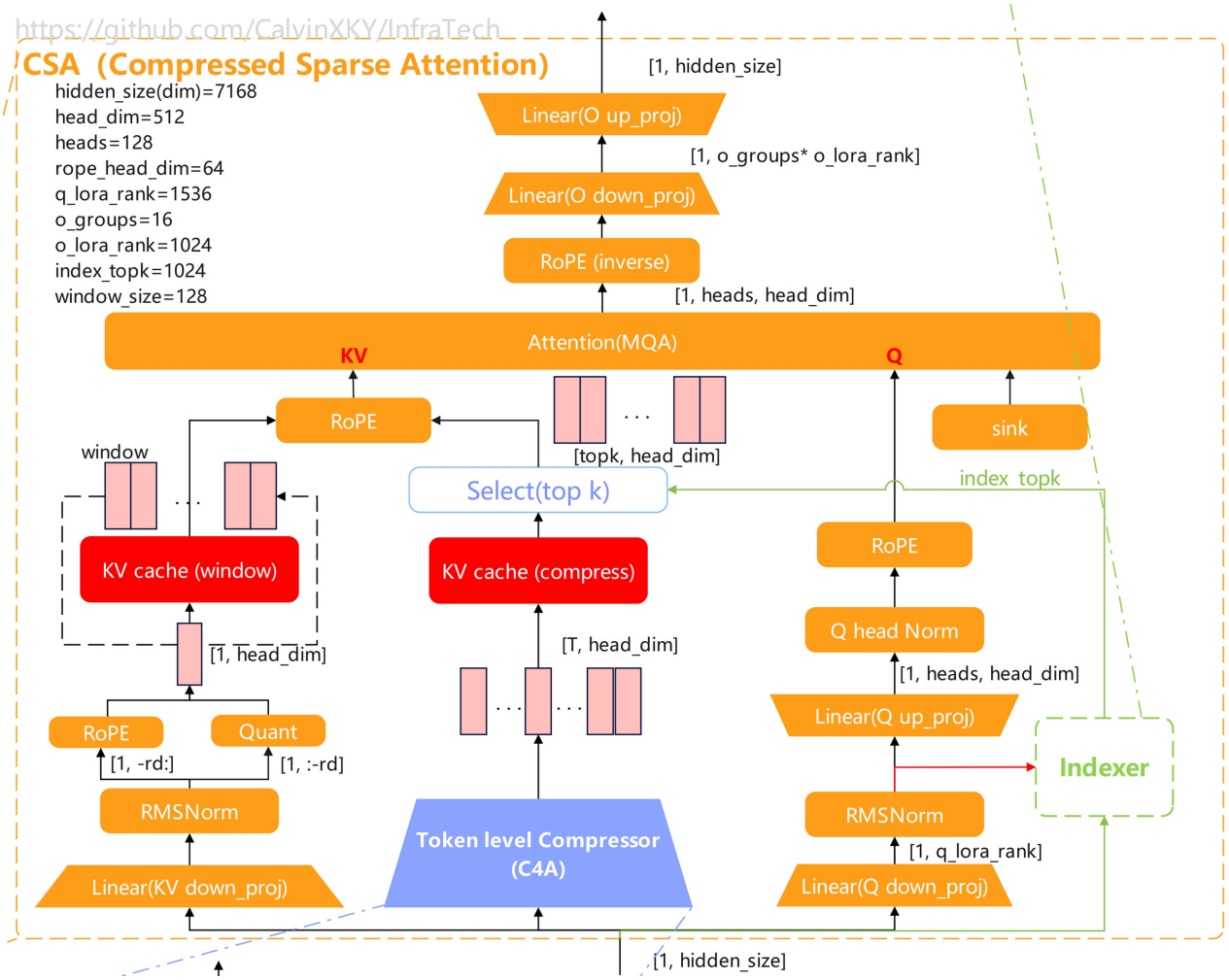


图 2 CSA 注意力层整体数据流（对应图 1 橙框）。最下方输入 x 分成三路：左侧生成局部 KV，中间经 Attention 的 Compressor 生成压缩 KV，右侧生成 attention query 并驱动右下角虚线框内的 Indexer（展开见图 7）。

`norm`，则是在此基础上再逐通道乘一个可学习向量 `weight`。因此我们先写无权重的 `rms_forward` 和 `rms_backward`，再拿它们搭出 `RMSNorm`。

5.1 无权重的 RMS：`rms_forward`

将输入 x 沿最后一维（长度记为 d）逐行归一化：

$$\texttt{inv_rms} = \left(\frac{1}{d}\sum_{c=0}^{d-1} x_c^2 + \varepsilon\right)^{-1/2},$$

$$y = x \cdot \texttt{inv_rms}.$$

函数返回 $(y, \texttt{cache})$，其中 `cache` 存什么由你自行设计，外部只会原封不动地把它传给 `rms_backward`。注意边界情况：当某一行全为 0 时，$\texttt{inv_rms} = \varepsilon^{-1/2}$ 依然有限，输出 y 应为全 0，不能除以零算出 NaN。

5.2 带权重的 RMSNorm

`RMSNorm` 直接复用 `rms_forward` 的结果，再逐通道乘上长度为 d 的参数向量 `weight`（初始全为 1）：

$$(y, \texttt{cache}) = \text{rms_forward}(x, \texttt{eps}),$$
$$\texttt{RMSNorm}(x) = \texttt{weight} \odot y.$$

5.3 反向

`rms_backward(dy, cache)` 根据上游梯度 $\mathrm{d}y$ 和 `cache` 算出同形状的 $\mathrm{d}x$；`RMSNorm.backward(dy)` 则在此之上还要把 `dweight` 累加进 `self._grads["weight"]`。

提示

- 注意 `inv_rms` 是整行 d 个元素共同决定的标量：对 x_c 求导时，除了直接来自 $x_c \cdot$ `inv_rms` 的 $\mathrm{d}y_c \cdot$ `inv_rms` 项外，别忘了 `inv_rms` 本身对 x_c 的导数项（它会把整行的 $\mathrm{d}y$ 和 x 耦合起来）。
- `RMSNorm` 的 `weight` 被所有前导维度共享，求 `dweight` 时要沿除最后一维外的所有维度求和。

6 框架提供的两个正交变换

架构图里的所有 RoPE 框对应 `csa/rope.py` 中的 `partial_rope`，Indexer 里的 Rotate 框则对应 `csa/ops.py` 中的 `rotate_activation`（图 3、图 4）。这两个函数前向直接调用即可，但写反向时必须清楚它们的转置是什么。

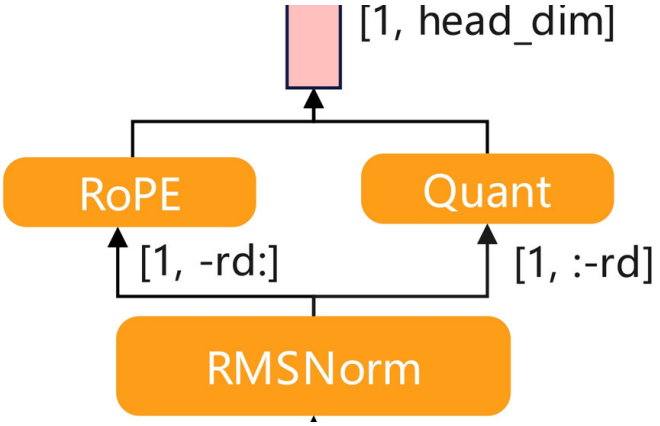


图 3 局部 KV 支路末尾的 partial RoPE（截自图 2）。`kv_norm` 输出后，只取末尾 d_r 个通道（`[1, -rd:]`）施加旋转位置编码，前面 $D - d_r$ 个通道保持不动，最后拼回完整的 `head_dim`。

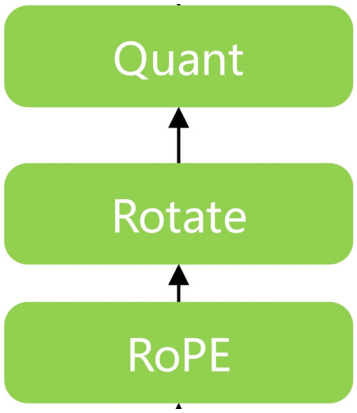


图 4 Index query 支路末尾依次经过 partial RoPE、Hadamard 旋转（Rotate）与量化钩子（截自图 7）。

6.1 部分旋转位置编码：`partial_rope`

DeepSeek 系列不把整条向量都做 RoPE，而是只旋转每个向量末尾的 d_r 个通道（d_r 为偶数，即 `rope_head_dim`），前面 $D - d_r$ 个通道原样保留。具体来说，`partial_rope(x, freqs, rd,`

`inverse=False`）从通道下标 $D - d_r$ 开始，把相邻两个通道配成一对 $(x_{D-d_r+2i}, x_{D-d_r+2i+1})$ $(0 \le i < d_r/2)$，在序列位置 t 上做二维平面旋转：

$$\begin{pmatrix} a \\ b \end{pmatrix} \mapsto \begin{pmatrix} a\cos\theta_{t,i} - b\sin\theta_{t,i} \\ a\sin\theta_{t,i} + b\cos\theta_{t,i} \end{pmatrix},$$
$$\theta_{t,i} = t\,\omega_i.$$

代码中预计算好的 `freqs_cis` 第 t 行存的就是复数 $e^{\mathrm{i}\theta_{t,i}} = \cos\theta_{t,i} + \mathrm{i}\sin\theta_{t,i}$；`partial_rope` 会把传入的 `freqs` 第 n 行作用在 `x[:, n]` 上，中间若有多余的头维度会自动广播。当传入 `inverse=True` 时，旋转角变为 $-\theta_{t,i}$（即把 sin 取反）。

下文把位置 t 上的正向旋转简记为线性算子 rope_t。由于二维旋转矩阵是正交矩阵，它的转置恰好等于逆旋转：

$$\mathrm{rope}_t^{\top} = \mathrm{rope}_t^{-1} = \mathrm{partial_rope}(\cdot,\ \texttt{freqs},\ d_r,\ \mathrm{inverse{=}True}).$$

因此反向传播非常直接：**前向做过正向 RoPE 的地方，反向对梯度调一次 `inverse=True`；前向做过逆 RoPE 的地方，反向调一次 `inverse=False`**。还要留心两类条目的位置约定：普通 token（attention query、index query、局部 KV 及注意力输出）处在位置 t，传入 `freqs_cis`$[:S]$；而第 j 个压缩条目代表从 mj 开始的一整块，位置取块起点 mj，传入步长为 m 的切片 `freqs_cis`$[0 : N \cdot m : m]$。

6.2 Hadamard 旋转：`rotate_activation`

在真实硬件上，Indexer 的打分用超低比特（FP4）矩阵乘加速；为了防止个别通道的离群大值撑爆量化范围，需要先乘一个正交矩阵把能量均匀摊到所有通道上。`rotate_activation(x)` 做的就是这件事：它沿最后一维（长度 d 必须为 2 的幂）做归一化的 Walsh–Hadamard 变换，记作 $\mathcal{H}x$。

因为归一化 Hadamard 矩阵同时满足对称与正交（$\mathcal{H}^{\top} = \mathcal{H}$ 且 $\mathcal{H}\mathcal{H} = I$），它带来两个很好的性质：一是 Indexer 的 query 和 key 两边同时左乘 $\mathcal{H}$ 后，内积 $(\mathcal{H}q)^{\top}(\mathcal{H}k) = q^{\top}\mathcal{H}^{\top}\mathcal{H}k = q^{\top}k$ 保持不变；二是 **$\mathcal{H}$ 的转置就是它自己**——反向传播经过 `rotate_activation` 时，对上游梯度再调一次 `rotate_activation` 即可。

7 Compressor

代码： 需完成 `Compressor.forward(x)` 和 `Compressor.backward(dentries)`。参数 `ape`、`wkv`、`wgate`、`norm`；属性 `head_dim`、`rope_head_dim`、`compress_ratio`、`rotate`、`freqs_cis`。

Compressor 的作用是把每 $m = 4$ 个连续 token 压成一个粗粒度 KV 条目，使远距离历史的条目数降到原来的 $1/m$。如果直接把每 m 个 token 硬切开各自池化，块与块之间就会有明显的边界割裂；如果改成步长为 m、窗口为 $2m$ 的普通滑窗，每个 token 又要被算两遍。CSA 用了一个很巧的折中：先投出 $2D_c$ 个通道（D_c 即 `self.head_dim`），把后半段 D_c 个通道留给当前块，前半段 D_c 个通道交由下一块使用。这样除了第 0 块外，每个压缩条目都跨着前后两个块共 $2m$ 个位置做门控池化（gated pooling），既消除了硬边界，又保证每个 token 的每个通道只参与一次池化。

Attention 层和 Indexer 各有一个 `Compressor` 实例，计算流程完全一样，只是配置不同：Attention 的 Compressor 输出维度 $D_c = D$、不做 Hadamard（`rotate=False`）；Indexer 的 Compressor 输出维度 $D_c = D_I$、末尾多一步 Hadamard（`rotate=True`）。

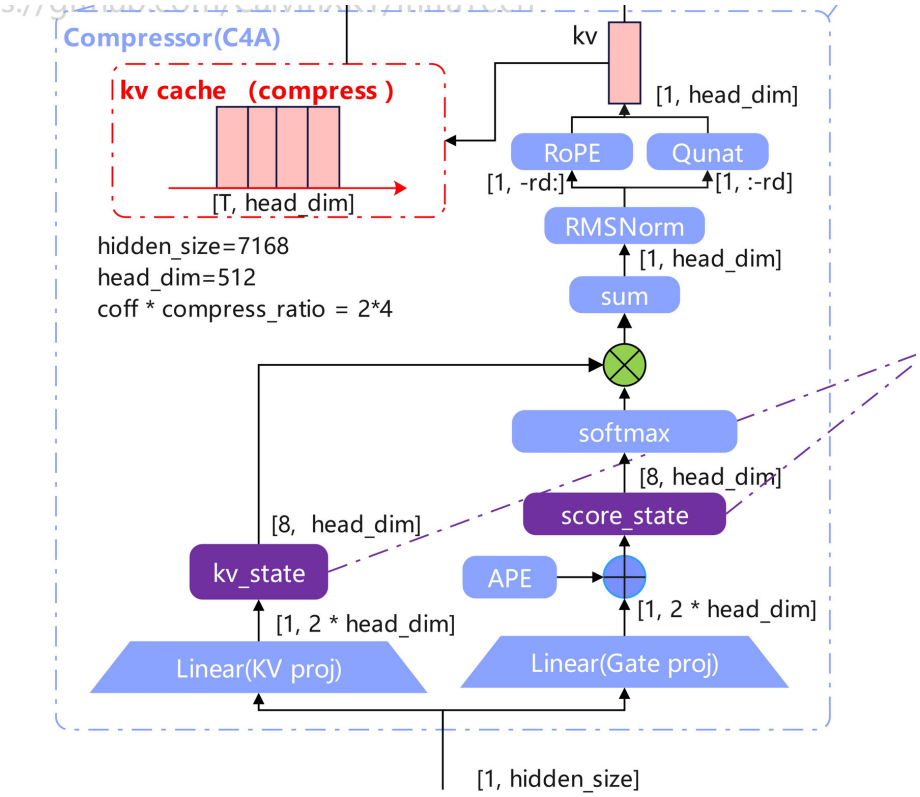


图 5 Compressor（C4A）结构（对应图 1 蓝框）。输入经 `wkv` 和 `wgate` 投出 $2D_c$ 维的 `kv` 与 `score`，加上块内位置偏置 `ape` 后，通过重叠排布拼成每个条目 $[2m, D_c]$ 的候选值 `values` 与门控打分 `logits`（即图中 `[8, head_dim]` 的 kv_state 与 score_state），再沿 $2m$ 个位置做逐通道 softmax 加权求和、`norm` 和 RoPE。Indexer 的 Compressor 在 RoPE 后还会再做一次 Hadamard 旋转。

7.1 投影与门控打分

先对输入做两个线性投影，宽度都是 $2D_c$（这里的 `kv` 是 Compressor 内部的投影，和第 3 节拼出的 `kv` 表不是同一个数组）：

$$\texttt{kv} = \texttt{wkv}(x) \in \mathbb{R}^{B\times S\times 2D_c},$$
$$\texttt{score} = \texttt{wgate}(x) \in \mathbb{R}^{B\times S\times 2D_c}.$$

长度为 S 的序列一共能凑出 $N = \lfloor S/m \rfloor$ 个完整块，正好覆盖前 $N \cdot m$ 个 token；凑不满一整块的尾部 token（下标 $\geq N \cdot m$）不参与压缩，只能通过局部滑动窗口被后续 token 读到。对前 $N \cdot m$ 个位置，按它在块内的相对位置 $r = t \bmod m$（即 $t = jm + r$，$0 \leq r < m$），给门控打分加上可学习的块内位置偏置 $\texttt{ape} \in \mathbb{R}^{m\times 2D_c}$：

$$\texttt{gate}_{b,t,:} = \texttt{score}_{b,t,:} + \texttt{ape}_{r,:}.$$

7.2 跨块重叠排布

接下来把 $2D_c$ 个通道对半劈开：前 D_c 个通道（$0 \le c < D_c$）留给下一个块 $j+1$，后 D_c 个通道（$D_c \le c < 2D_c$）归当前块 j。于是第 j 个压缩条目在每个通道 $c \in [0, D_c)$ 上，都汇集了来自块 $j-1$ 和块 j 的 $2m$ 个位置。为了方便后面做 softmax 和加权求和，把每个条目要用的这 $2m$ 个位置单独排成两个形状为 $[B, N, 2m, D_c]$ 的数组：

- `values`：要被加权平均的值，从 `kv` 中取；
- `logits`：每个值对应的门控打分，从上一小节的 `gate` 中取，后面对它做 softmax 得到权重。

下标 $p = 0, \ldots, 2m-1$ 是条目内的位置：前 m 个（$p < m$）来自前一块 $j-1$，后 m 个（$p \ge m$）来自本块 j。这两个数组只是把已有的数重新排列，图 5 里的 kv_state 和 score_state 就是它们。代码里没有现成的这两个变量，需要按下式用切片自己拼出来：

$$\texttt{values}_{b,j,p,c} = \begin{cases} \texttt{kv}_{b,\,(j-1)m+p,\,c}, & p < m,\ j \ge 1 \quad (\text{取块} j-1 \text{ 的前半通道}), \\ 0, & p < m,\ j = 0, \\ \texttt{kv}_{b,\,jm+p-m,\,D_c+c}, & p \ge m \quad (\text{取块} j \text{ 的后半通道}), \end{cases}$$

$$\texttt{logits}_{b,j,p,c} = \begin{cases} \texttt{gate}_{b,\,(j-1)m+p,\,c}, & p < m,\ j \ge 1, \\ -\infty, & p < m,\ j = 0, \\ \texttt{gate}_{b,\,jm+p-m,\,D_c+c}, & p \ge m. \end{cases}$$

第 0 个条目没有前一块，所以前 m 个位置的门控 logit 设为 $-\infty$，softmax 后权重自然为 0，实际上只池化块 0 自己的 m 个位置。反过来看，除最后一个完整块的前半通道（本来要留给块 N，但块 N 还没凑齐）被丢弃外，前 $N \cdot m$ 个 token（即 N 个完整块里的全部 token）的其余通道都恰好落进唯一一个条目里（图 6）。

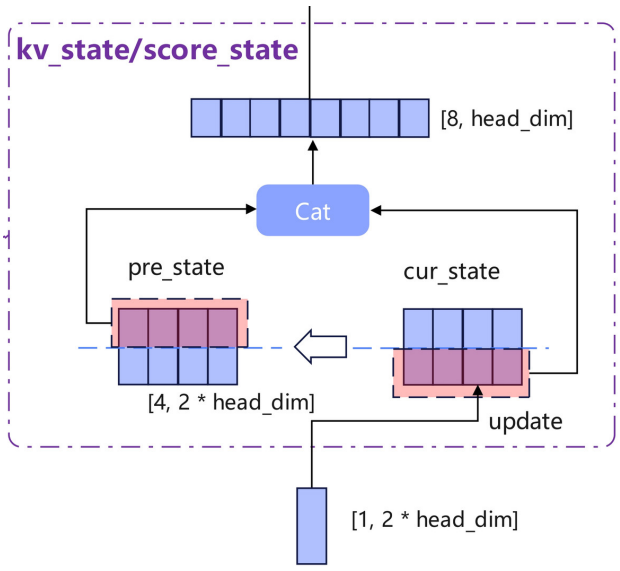


图 6 跨块重叠排布（对应图 1 紫框）。取前一块 $j-1$（pre_state）的前半通道与当前块 j（cur_state）的后半通道，沿位置维拼接成 $[2m, D_c]$ 的 $\texttt{values}_{b,j}$ 和 $\texttt{logits}_{b,j}$。

7.3 逐通道 softmax 池化

有了 $[B, N, 2m, D_c]$ 的候选值和门控打分，在每个通道 c 上独立地沿 $2m$ 个位置做 softmax 加权求和：

$$\texttt{alpha}_{b,j,p,c} = \frac{\exp \texttt{logits}_{b,j,p,c}}{\sum_{q=0}^{2m-1} \exp \texttt{logits}_{b,j,q,c}},$$
$$\texttt{pooled}_{b,j,c} = \sum_{p=0}^{2m-1} \texttt{alpha}_{b,j,p,c}\, \texttt{values}_{b,j,p,c}.$$

实现时照常先减去该通道 $2m$ 个位置的最大值再取指数，避免溢出。

7.4 归一化与位置旋转

池化后的 $\texttt{pooled} \in \mathbb{R}^{B\times N\times D_c}$ 先过跨通道的 RMSNorm（`self.norm`），再按该块起始 token 的位置 mj 做 partial RoPE；若 `self.rotate` 为真，最后再做一次 Hadamard 旋转：

$$\texttt{entries}_{b,j} = \begin{cases} \mathcal{H}\ \mathrm{rope}_{mj}\big(\texttt{norm}(\texttt{pooled}_{b,j})\big), & \texttt{rotate} = \mathrm{True}, \\ \mathrm{rope}_{mj}\big(\texttt{norm}(\texttt{pooled}_{b,j})\big), & \texttt{rotate} = \mathrm{False}. \end{cases}$$

注意这里的 RoPE 位置取的是块起点 mj（即传入 $\texttt{freqs_cis}[0 : N\cdot m : m]$），而不是块末尾。当序列太短、连一个完整块都凑不够（$S < m$，即 $N = 0$）时，`forward` 直接返回 `None`。

7.5 一个数值例子

不妨设 $m = 4$，所有门控 logit 相等（此时 softmax 退化为等权平均），单独看某一个通道 c：假设块 0 在后半通道 $D_c + c$ 上的 4 个值为 $[1, 3, 5, 7]$，在前半通道 c 上为 $[2, 4, 6, 8]$；块 1 在后半通道 $D_c + c$ 上为 $[10, 12, 14, 16]$。那么条目 0 只对块 0 的后半通道求平均，得到 $\texttt{pooled}_{0,c} = (1+3+5+7)/4 = 4$；条目 1 则把块 0 的前半通道与块 1 的后半通道拼起来求平均，得到 $\texttt{pooled}_{1,c} = (2+4+6+8+10+12+14+16)/8 = 9$。当门控 logit 不均匀时，只是把这里的等权平均换成每个通道各自算出的 $2m$ 个 softmax 权重。

7.6 反向

`backward(dentries)` 接收 $\texttt{dentries} \in \mathbb{R}^{B\times N\times D_c}$（若 $N = 0$ 则形状为 $[B, 0, D_c]$），返回对输入的梯度 $\mathrm{d}x \in \mathbb{R}^{B\times S\times E}$，并把 `wkv`、`wgate`、`ape`、`norm` 四处参数的梯度累加到位。

规格与提示

- 先逆向穿过后处理：Hadamard 的转置是它自身，RoPE 的转置是逆旋转（第 6 节），随后调用 `norm.backward` 得到 dpooled。
- 池化式 $\texttt{pooled} = \sum_p \texttt{alpha}_p \texttt{values}_p$ 同时依赖 `values` 和经由 softmax 算出的 `alpha`，所以对 `values` 和 `logits` 两边都要算梯度。
- 跨块排布本身只是切片搬运：反向时把 dvalues 和 dlogits 的前后两半原路放回 dkv、dscore 对应的块和通道即可；未进入任何条目的最后一块前半通道及尾部 token 梯度为 0。
- `ape` 在所有完整块上重复使用，它的梯度需要跨 batch 和所有 N 个块求和。

8 Indexer

代码： 需完成 `Indexer.forward(x, qr, offset)` 和 `Indexer.backward_scores(dscore)`。参数 `wq_b`、`weights_proj` 和独立的 `compressor`；属性 `n_heads` $= H_I$、`head_dim` $= D_I$、`index_topk` $= k$、`softmax_scale` $= D_I^{-1/2}$。

序列里有 $N = \lfloor S/m \rfloor$ 个压缩块，每个 query t 只挑其中最相关的至多 k 个来读。为了把“挑块”这件事做得尽量轻，Indexer 直接复用 Attention 层算好的低秩表示 `qr`，投出头数更少（H_I）、维度更低（D_I）的 index query，和它自己的 Compressor 产出的 `keys` 做点积打分，最后在因果掩码下取 top-k（图 7）。

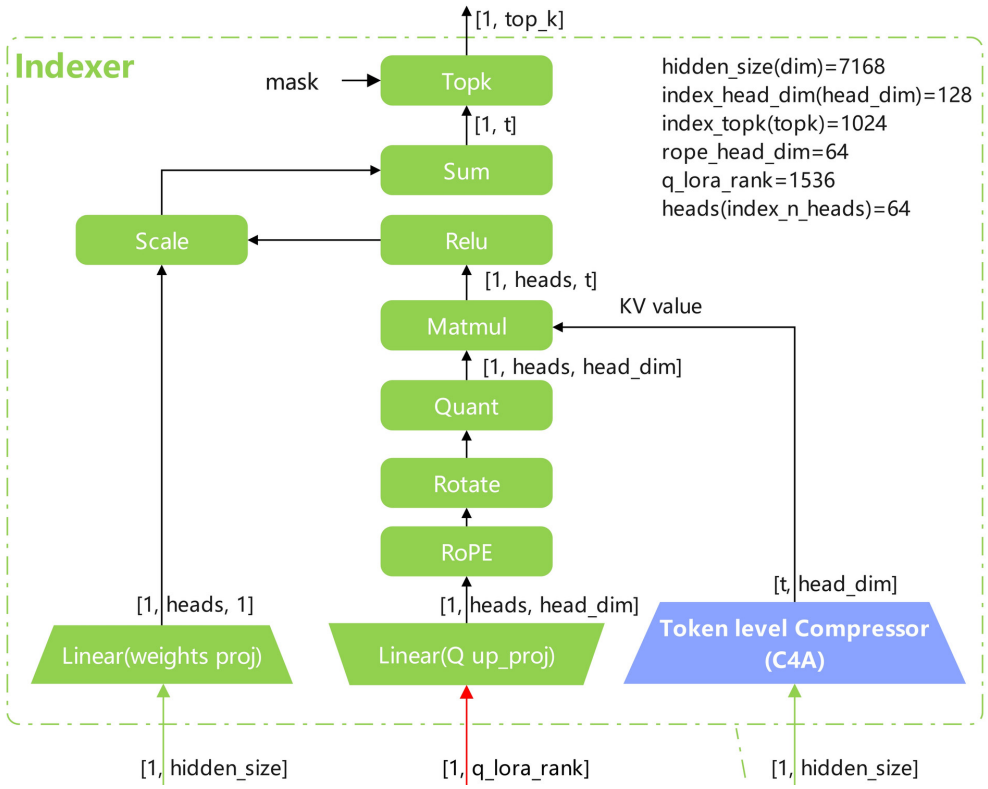


图 7 Indexer 数据流（对应图 1 绿框）。右侧将共享低秩表示 `qr` 经 `wq_b`、RoPE 与 Hadamard 旋转变成 index query `q`；左侧用 Indexer 自己的 Compressor 将 x 压成 N 个粗粒度 `keys`；两者点积过 ReLU 后，按 `weights_proj` 算出的每头权重加权求和得到打分矩阵，经因果掩码（mask）后取 top-k 块号。

8.1 Index query、key 与每头权重

记 `freqs` $=$ `freqs_cis`$[:S]$，三路输入分别计算如下：

$$\begin{aligned}
\texttt{q} &= \mathcal{H}\ \mathrm{rope}_t\big(\mathrm{reshape}(\texttt{wq_b}(\texttt{qr}), [B, S, H_I, D_I])\big), \\
\texttt{keys} &= \texttt{compressor}(x) \in \mathbb{R}^{B\times N\times D_I} \qquad (N = 0 \text{ 时视为形状 } [B, 0, D_I] \text{ 的空数组}), \\
\texttt{weights} &= \texttt{weights_proj}(x) \cdot \texttt{softmax_scale} \cdot H_I^{-1/2} \in \mathbb{R}^{B\times S\times H_I}.
\end{aligned}$$

和 attention query 相比，index query `q` 省去了每头的无权重 RMS 归一化，但末尾多了一步 Hadamard 旋转 $\mathcal{H}$（从而与开启了 `rotate=True` 的 `keys` 处在同一个旋转空间里）。另外注意，每头权重 `weights`

只是线性层输出乘上缩放常数 $D_I^{-1/2}H_I^{-1/2}$，**不做 softmax**，因此数值正负皆可。

8.2 多头 ReLU 打分

Indexer 的每个头先算 query t 与压缩块 j 的点积，过一次 ReLU 截掉负响应，再按该 token 在各头上的权重 $\texttt{weights}_{b,t,h}$ 加权求和：

$$\texttt{dots}_{b,t,h,j} = \sum_c \texttt{q}_{b,t,h,c}\,\texttt{keys}_{b,j,c},$$
$$\texttt{score}_{b,t,j} = \sum_h \texttt{weights}_{b,t,h}\,\max(\texttt{dots}_{b,t,h,j}, 0).$$

注意 ReLU 作用在每个头的点积上，不能先把各头点积加起来再做 ReLU。

8.3 因果掩码与 top-k 选块

第 j 个压缩块覆盖了 token $jm, jm+1, \ldots, (j+1)m-1$。按因果性，只有当这一整块的最后一个 token 已经出现（即 $(j+1)m-1 \le t$）时，位置 t 的 query 才允许读取块 j。换算过来，位置 t 可读的完整块数量为 $\texttt{ready}_t = \lfloor (t+1)/m \rfloor$，即只允许读编号 $j < \texttt{ready}_t$ 的块。在调 top-k **之前**，先把所有不可读块的分数抹成 $-\infty$，并把掩码后的分数矩阵保存在 `self.last_scores`（形状 $[B, S, N]$）中：

$$\texttt{last_scores}_{b,t,j} = \begin{cases} \texttt{score}_{b,t,j}, & j < \texttt{ready}_t, \\ -\infty, & j \ge \texttt{ready}_t. \end{cases}$$

接着令 $K_c = \min(k, N)$，对每行调用 `topk_lower_index_first` 选出分数最高的 K_c 个块号 $\texttt{ids}_{b,t,0}, \ldots, \texttt{ids}_{b,t,K_c-1}$（分数相同时编号小的排前面）。

这里有一个边界细节：序列开头的 query 可读块数 $\texttt{ready}_t$ 往往不足 K_c 个，但 top-k 函数依然会凑满 K_c 个下标返回——多出来的那些其实落在了分数为 $-\infty$ 的不可读块上。所以最后要把所有 $\texttt{ids}_{b,t,r} \ge \texttt{ready}_t$ 的无效结果改写成 -1，而对有效块号则加上调用方传入的偏移量 `offset`（prefill 中 Attention 的 `kv` 表前 S 行是局部条目，因此 `Attention` 会传入 $\texttt{offset} = S$）：

$$\texttt{topk_idxs}_{b,t,r} = \begin{cases} \texttt{ids}_{b,t,r} + \texttt{offset}, & \texttt{ids}_{b,t,r} < \texttt{ready}_t, \\ -1, & \text{否则}. \end{cases}$$

千万不要先对未掩码的 `score` 取 top-k 再把未来块过滤成 -1——那样高分的未来块会先把真正可读的历史块挤出前 K_c 名。

8.4 反向

由于整数下标选块不可导，常规的 `Attention.backward` 不会把梯度传进 Indexer，Indexer 专属参数的梯度保持为 0。但在实际训练或辅助损失下，外部可以直接给出损失对 `self.last_scores` 的梯度 $\mathrm{dscore} \in \mathbb{R}^{B\times S\times N}$。`backward_scores(dscore)` 接收这个 dscore，返回 $(\mathrm{d}x, \mathrm{dqr})$，并累加 Indexer 内部 `wq_b`、`weights_proj` 和 `compressor` 的参数梯度。被因果掩码屏蔽的位置（$j \ge \texttt{ready}_t$）不传梯度。

提示

- 先把 $j \geq \texttt{ready}_t$ 处的 dscore 置零；$\max(\texttt{dots}, 0)$ 对 dots 的导数就是指示函数 $\mathbb{I}[\texttt{dots} > 0]$。
- 前向算 weights 时乘了常数 $\texttt{softmax_scale} \cdot H_I^{-1/2}$，回传给 weights_proj.backward 前别忘了把这个常数乘到 dweights 上。
- q 依次做过 RoPE 和 Hadamard，回传时先调一次 Hadamard，再调逆 RoPE（第 6 节）；dkeys 直接交给 self.compressor.backward，它算出的输入梯度与 weights_proj.backward 的结果相加即为 dx。

9 稀疏注意力 sparse_attn

代码： 需完成 sparse_attn(q, kv, attn_sink, topk_idxs, softmax_scale, return_cache=False) 和 sparse_attn_backward(dout, cache)。

拿到路由表 topk_idxs 后，sparse_attn 让每个位置的 H 个 query 头去共享 kv 表里抓出对应的行，做带 attention sink 的多查询注意力（图 8）。

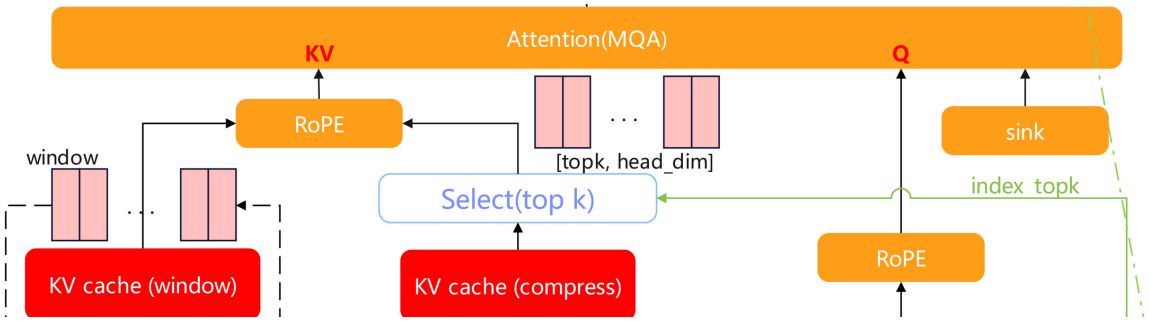


图 8 稀疏注意力（截自图 2）。每个位置把窗口内的局部条目与 Select(top k) 挑出的压缩条目合在一起作为候选 KV，供 H 个 query 头共享读取，并引入每头一个可学习的 sink logit。图中 KV 上方的 RoPE 框代表各条目已预先在自身位置上做过旋转，sparse_attn 内部不再做 RoPE。

9.1 前向

函数接收 attention query $\texttt{q} \in \mathbb{R}^{B\times S\times H\times D}$、共享 KV 表 $\texttt{kv} \in \mathbb{R}^{B\times N_{kv}\times D}$、每头一个可学习标量 $\texttt{attn_sink} \in \mathbb{R}^{H}$，以及槽位表 $\texttt{topk_idxs} \in \mathbb{Z}^{B\times S\times K}$（其中 -1 表示空槽位）。对固定的 (b,t)，只看 $\texttt{topk_idxs}_{b,t}$ 中值非负的有效槽位 r，记它从 kv 第 b 个样本里取出的行向量为 $a_r = \texttt{kv}_{b,\,\texttt{topk_idxs}_{b,t,r}} \in \mathbb{R}^{D}$。在每个注意力头 h 上计算：

$$
\begin{aligned}
s_{h,r} &= \texttt{softmax_scale} \cdot \texttt{q}_{b,t,h} \cdot a_r, \\
Z_h &= \exp \texttt{attn_sink}_h + \sum_u \exp s_{h,u}, \\
p_{h,r} &= \frac{\exp s_{h,r}}{Z_h}, \\
\texttt{out}_{b,t,h} &= \sum_r p_{h,r}\, a_r .
\end{aligned}
$$

可以把 $\texttt{attn_sink}_h$ 理解成一个“虚拟 KV 条目”：它的 logit 是 $\texttt{attn_sink}_h$，和所有真实条目一起进入分母 Z_h，分走的概率是

$$p_{h,\text{sink}} = \frac{\exp \texttt{attn_sink}_h}{Z_h} = 1 - \sum_r p_{h,r}.$$

但它的 value 是零向量，所以 $\texttt{out}_{b,t,h}$ 里没有这一项。sink 的作用就是把真实条目的权重之和压到 1 以下：当某个头在当前位置没有特别想看的条目时，权重可以分流到 sink 上，输出随之变小。前向用不到 $p_{h,\text{sink}}$，但反向求 dattn_sink 时要用到它。

实现时注意四点：

- **K=V 且跨头共享：**同一个向量 a_r 既和 $\texttt{q}_{b,t,h}$ 做点积当 key，又在加权求和里当 value；同一位置的 H 个头读的是同一批 a_r，只是各自算出不同的注意力权重 $p_{h,r}$。
- **统一归一化：**局部窗口槽位和压缩块槽位放在同一个 softmax 里归一化，不能分成两组各做各的 softmax。
- **重复槽位与全空行：**如果 $\texttt{topk_idxs}_{b,t}$ 里有两个槽位指向同一个行号，它们当作两个独立槽位各自进分母、各自加权；如果整行全为 -1，分母只剩 $\exp \texttt{attn_sink}_h$，输出 $\texttt{out}_{b,t,h}$ 自然为全 0。
- **数值稳定：**取指数前先减去该头所有有效槽位与 sink 的最大值 $\max\big(\texttt{attn_sink}_h, \max_r s_{h,r}\big)$（若整行无有效槽位则最大值就是 $\texttt{attn_sink}_h$）。

当 `return_cache=True` 时返回 `(out, cache)`，否则只返回 `out`。

9.2 反向

`sparse_attn_backward(dout, cache)` 根据上游梯度 dout 和前向保存的 cache，算出 dq、dkv 和 dattn_sink，形状分别与 q、kv、attn_sink 相同。

规格与提示

- 推导 softmax 反向时，把 sink 看成第 $K+1$ 个候选（其 value 为 0，故 $\mathrm{d}p_{h,\text{sink}} = 0$），套用第 2.1 节例子里的 $\mathrm{d}s_i = p_i(\mathrm{d}p_i - \sum_j p_j \mathrm{d}p_j)$ 即可，无需写出雅可比矩阵。
- 因为同一个条目 a_r 既当 value 又当 key，它收到的梯度包含两项，而且要对所有 H 个头求和。
- 同一个 kv 条目会被不同位置的 query 反复读取，甚至在同一行的不同槽位里重复出现；往 dkv 写回时必须用 `np.add.at` 累加，不能直接用下标赋值。
- dattn_sink 是把所有 (b,t) 位置上 sink logit 的梯度加起来。

10 整层串联 Attention

代码：需完成 `Attention.forward(x)` 和 `Attention.backward(dy, dindex_scores=None)`。

前面几个模块都就绪后，`Attention` 负责把它们拼成完整的一层。

10.1 前向

记 `freqs` = `freqs_cis`$[:S]$，本节简写的 rope$(\cdot)$ 均指在位置 $0,\dots,S-1$ 上调用 `partial_rope(·, freqs, rope_head_dim)`。

第一步：算 attention query 与局部 KV（图 9、图 10）。

$$\begin{aligned}
\texttt{qr} &= \texttt{q_norm}(\texttt{wq_a}(x)) \in \mathbb{R}^{B\times S\times R},\\
\texttt{q},\ \texttt{q_rms} &= \text{rms_forward}\big(\text{reshape}(\texttt{wq_b}(\texttt{qr}),[B,S,H,D]),\ \texttt{eps}\big),\\
\texttt{q} &\leftarrow \text{rope}(\texttt{q}),\\
\texttt{local} &= \text{rope}\big(\texttt{kv_norm}(\texttt{wkv}(x))\big) \in \mathbb{R}^{B\times S\times D}.
\end{aligned}$$

其中 `q_rms` 是 `rms_forward` 返回的缓存，留给反向使用。

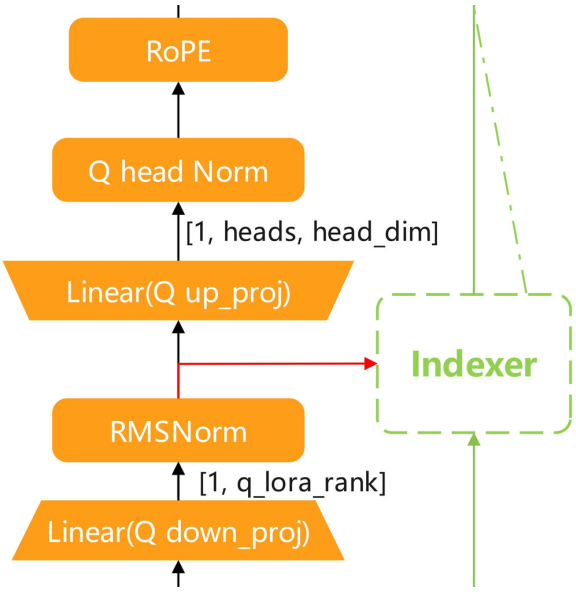


图 9 Query 支路（截自图 2）。x 经 `wq_a` 和 `q_norm` 得到低秩表示 `qr`（红线同时分给 Indexer），再经 `wq_b`、每头无权重 `rms_forward` 和 RoPE 得到 attention query `q`。

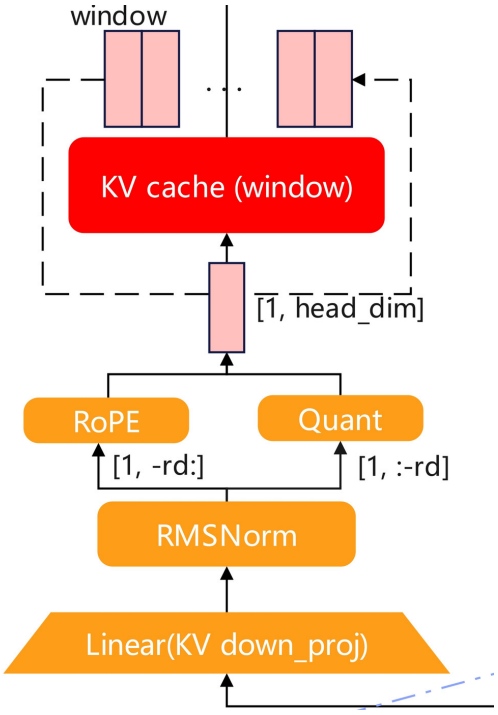


图 10 局部 KV 支路。x 经 `wkv`、`kv_norm` 和 RoPE 得到每个 token 一行的 `local` $\in \mathbb{R}^{B\times S\times D}$，供滑动窗口读取。

第二步：拼路由表 `topk_idxs` 与完整 `kv` 表。每个位置 t 的滑动窗口最多看最近 w 个 token（含 t 自身），因此窗口槽位宽度取 $K_w=\min(S,w)$，第 r 个槽位（$0\le r<K_w$）为：

$$\texttt{window}_{t,r}=\begin{cases}\max(0,t-w+1)+r, & \max(0,t-w+1)+r\le t,\\ -1, & \text{否则.}\end{cases}$$

当 $t\ge w-1$ 时，这 K_w 个槽位正好是连续的 $t-w+1,\dots,t$；而开头几个 token（$t<w-1$）左侧历史不足 K_w 个，超出 t 的尾部槽位填 -1。将窗口槽位（广播到 batch 维）与 Indexer 返回的 $K_c=\min(k,N)$ 个压缩槽位沿最后一维拼起来，再把 `local` 与 Attention 的 Compressor 输出的条

目沿第 1 维拼起来：

$$\texttt{topk_idxs} = \big[\texttt{window} \;;\; \texttt{indexer}(x, \texttt{qr}, S)\big] \in \mathbb{Z}^{B\times S\times(K_w+K_c)},$$
$$\texttt{kv} = \big[\texttt{local} \;;\; \texttt{compressor}(x)\big] \in \mathbb{R}^{B\times(S+N)\times D}.$$

当 $S < m$（即 $N = 0$）时，Attention 的 Compressor 返回 `None`，此时 `kv` 就是 `local` 本身。拼好的 `topk_idxs` 记得赋给 `self.last_topk_idxs`。

第三步：稀疏注意力、逆 RoPE 与分组输出投影（图 11）。用缩放系数 `softmax_scale` $= D^{-1/2}$ 调 `sparse_attn` 得到注意力输出 o：

$$o = \text{sparse_attn}\big(\texttt{q}, \texttt{kv}, \texttt{attn_sink}, \texttt{topk_idxs}, \texttt{softmax_scale}\big),$$
$$\texttt{grouped} = \text{reshape}\big(\text{rope}^{-1}(o),\ [B, S, G, HD/G]\big),$$
$$W^a = \text{reshape}\big(\texttt{wo_a.weight},\ [G, R_o, HD/G]\big),$$
$$\texttt{reduced}_{b,s,g,r} = \sum_d \texttt{grouped}_{b,s,g,d}\, W^a_{g,r,d},$$
$$y = \texttt{wo_b}\big(\text{reshape}(\texttt{reduced}, [B, S, GR_o])\big).$$

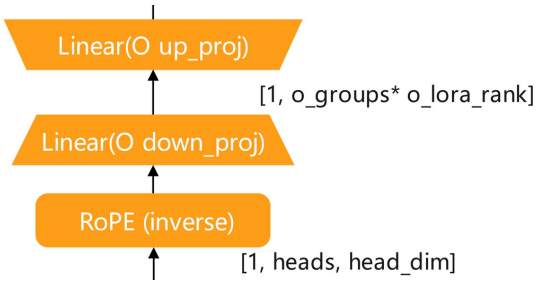


图 11 输出投影支路（截自图 2）。注意力输出 o 先做一次逆 RoPE，再把 H 个头均分成 G 组，每组用 `wo_a` 压到低秩 R_o，最后经 `wo_b` 还原回模型维度 E。

这里有两个设计细节值得留意：

- **为什么输出 o 要做一次逆 RoPE？**在普通注意力里，RoPE 只加在 Q 和 K 上，V 不做旋转；但 CSA 的 MQA 让同一个向量既当 K 又当 V，我们在造 `local` 和 `compressed` 时已经把各自位置 τ 的旋转 rope_τ 乘进去了。结果加权求出来的 $o_{b,t,h}$ 末尾 d_r 个通道还带着绝对位置旋转 rope_τ。在输出端补乘一次 rope_t^{-1}，刚好利用旋转群性质把绝对旋转变成相对位置旋转 $\text{rope}_{\tau-t}$。
- **分组投影 `wo_a`：**为了省参数，输出端没有直接用 $[E, HD]$ 的大矩阵，而是把相邻的 H/G 个头并成一组（共 G 组，每组宽 HD/G），用形状为 $[GR_o, HD/G]$ 的 `wo_a.weight` 把每组独立压到 R_o 维，拼成 GR_o 维后再过 `wo_b`。因为 `wo_a` 做的是按组矩阵乘（分组 einsum），前向不要直接调 `self.wo_a(...)`，反向也要手动算出 dgrouped 和 `self.wo_a._grads["weight"]`。

10.2 反向

`backward(dy, dindex_scores=None)` 接收 $\mathrm{d}y \in \mathbb{R}^{B\times S\times E}$，返回 $\mathrm{d}x$，并把层内所有子模块（含 Attention 的 Compressor，以及传入 `dindex_scores` 时的 Indexer）的参数梯度累加到位。

规格与提示

- 按前向相反顺序依次回传：`wo_b` → 分组的 `wo_a` → 逆 RoPE → `sparse_attn`，随后拆到 query 支路和两条 KV 支路。
- 前向在 o 上做的是逆 RoPE，所以反向对 `dgrouped` 还原成 $[B, S, H, D]$ 后要做**正向** RoPE 才得到 do；反过来，前向在 `q` 和 `local` 上做的是正向 RoPE，反向就要做**逆** RoPE。
- `sparse_attn_backward` 返回的 `dkv` 前 S 行属于 `local`，后 N 行属于 Attention 的 Compressor（当 $N > 0$ 时，交给 `self.compressor.backward`）。
- 最终的 dx 要把局部 KV、Attention 的 Compressor、共享 query（即 `wq_a`）这三路算出的输入梯度全部加起来；如果传入了非空的 `dindex_scores`，还要把 `self.indexer.backward_scores` 返回的 dx_I 和 d`qr`$_I$ 分别汇入 dx 和 d`qr`。

11 用一条短序列走一遍前向

为了把前面的槽位编号和因果边界彻底对齐，我们拿一条 $B = 1, S = 9, m = 4, w = 3, k = 1$ 的短序列手算一遍路由表。

- 序列长 $S = 9$，一共凑出 $N = \lfloor 9/4 \rfloor = 2$ 个完整压缩块：条目 0 池化 token 0–3 的后半通道，RoPE 位置取 0，从第 3 个 token 起可读（因为块 0 末尾是 token 3，`ready`$_3 = \lfloor 4/4 \rfloor = 1$）；条目 1 池化 token 0–3 的前半通道与 token 4–7 的后半通道，RoPE 位置取 4，从第 7 个 token 起可读（`ready`$_7 = 2$）。
- 末尾的 token 8 凑不满第 2 块，不进入任何压缩条目，但能通过滑动窗口被自己看到。

拼接后的 `kv` 表长 $S + N = 11$ 行：[`local`$_0, \ldots,$ `local`$_8$, `compressed`$_0$, `compressed`$_1$]，所以两个压缩条目在 `kv` 中的行号分别是 9 和 10（即块号加 `offset` $= 9$）。假定 Indexer 在 query 7 上对两个压缩块算出的分数为 $(0.2, 1.3)$，在 query 8 上为 $(2.0, 1.0)$，几个典型位置的槽位如下：

query t	可读块数 `ready`$_t$	窗口槽位（$w = 3$）	Indexer 槽位（$k = 1$）	拼接后的 `topk_idxs`$_t$
0	0	$[0, -1, -1]$	$[-1]$	$[0, -1, -1, -1]$
2	0	$[0, 1, 2]$	$[-1]$	$[0, 1, 2, -1]$
3	1	$[1, 2, 3]$	$[9]$	$[1, 2, 3, 9]$
6	1	$[4, 5, 6]$	$[9]$	$[4, 5, 6, 9]$
7	2	$[5, 6, 7]$	$[10]$	$[5, 6, 7, 10]$
8	2	$[6, 7, 8]$	$[9]$	$[6, 7, 8, 9]$

可以看到：query 0 和 query 2 处还没有任何完整块生成（`ready`$_t = 0$），`last_scores` 整行都是 $-\infty$；虽然 `topk_lower_index_first` 仍会返回下标 0，但因为 $0 \not< $ `ready`$_t$，最终被替换为 -1。而在 query 7 处，每个注意力头会取出 `local`$_5$, `local`$_6$, `local`$_7$ 和 `compressed`$_1$（第 10 行）共四个向量，连同该头的 `attn_sink`$_h$ 一起做 5 项的 softmax，对四个向量加权求和后，在位置 7 上做逆 RoPE，最后送入分组输出投影。

第二阶段：片上内存受限的融合 kernel

12 存储模型与评分规则

代码： 框架 handout/csa/sim.py 和 handout/csa/kernels.py；需完成 handout/csa/fused.py 的宿主函数和它们启动的 kernel。

第一阶段用整块 NumPy 数组一口气算完，中间的注意力权重、池化门控和打分矩阵全都直接落进内存。到了真实 GPU 上，这种写法会把时间全花在显存（HBM）和片上高速缓存（SRAM / 寄存器）之间的来回搬运上，甚至直接撑爆显存。因此第二阶段引入一个两级存储模拟器 `csa/sim.py`，让你按类似 Triton / TileLang 的块级（tile-level）方式重写核心算子：

- **全局显存（HBM）：** 由 `sim.Tensor` 表示。宿主函数（host function）只能调用 `sim.alloc` 分配输出张量、调用 `sim.launch` 启动 kernel，或者用 `t.reshape(...)` 给同一块显存换个视图（零拷贝、不计流量），本身绝不能直接读写张量里的数值。
- **片上内存（SRAM）：** `sim.launch(kernel, grid, *args)` 会对 `grid` 中的每个坐标 `pid` 调一次 `kernel(pid, *args)`。每次调用称为一个 **block**，独享一份 **48 KiB** 的片上内存。block 内创建的所有数组（包括读入的块、算出的中间结果、临时数组乃至标量）只要还被引用，就一直占用当前 block 的片上额度（切片、转置等视图与原数组共享内存，不重复计费）；一旦实时占用超过 48 KiB，立刻抛出 `OnChipMemoryError`。block 运行结束时，其片上数据全部销毁。

为了与第一阶段高精度比对，底层数值仍用 `float64` 计算，但**容量和显存流量一律按每个元素 4 字节**（对应真实硬件上的 FP32 / INT32）记账。kernel 内部可以使用的原语如下：

调用	作用与流量记账
`load(t, index)`	把 `t[index]` 拷到片上（`index` 仅限整数和切片）；按读入元素数计读流量
`gather(t, ids, prefix)`	按片上整数下标数组 `ids` 从 `t[prefix]` 收集行，返回片上的 `(rows, valid)`；下标为 -1 处返回零行且不读显存，只对有效行计读流量
`store(t, index, x)`	把片上数组 `x` 写回 `t[index]`；按写入元素数计写流量
`atomic_add(t, index, x)`	原子累加 `t[index] += x`，允许多个 block 并发写同一位置；计入原子流量
`scatter_add(t, ids, x, prefix)`	对每个 $\texttt{ids}_i \geq 0$ 执行原子散射累加 `t[prefix][ids[i]] += x[i]`（重复下标依次累加）；只对有效行计原子流量
`zeros`、`ones`、`full`、`arange`	在当前 block 的片上内存中新建数组
`merge_topk(s, i, s2, i2, k)`	在片上合并两组候选 `(s, i)` 与 `(s2, i2)`，按“分数高优先、同分编号小优先”每行保留前 k 个
`capacity()`、`record()`	查询片上容量上限；`with sim.record() as launches:` 可在自测时记录每次 launch 的流量和片上峰值

编程规则。宿主代码不得读取张量数值；kernel 内只能通过上表原语生成片上数组，不能直接拿普通 NumPy 数组参与运算；片上数据（含标量）只能通过 `store`、`atomic_add`、`scatter_add` 写回显存，

不得带出 block；严禁调用框架专用的 `to_host`、`to_device`、`configure`，严禁访问下划线开头的私有属性或使用 `vars`、`getattr`、`setattr`、`globals` 等反射手段。

评分标准。每个宿主函数必须先保证计算结果与第一阶段一致、且没有任何 block 超出 48 KiB 片上容量；在此前提下，再检查两道硬指标：

$$T = \text{读字节} + \text{写字节} + \text{原子字节},$$
$$T_{\text{学生}} \le 1.5\, T_{\text{参考}} + 4096,$$

以及前向传给反向的 `saved` 额外张量大小上限：`sparse_attn` 除输入和 `out` 外，每个 (b, t, h) 最多存 4 个数；`gated_pool` 除输入和 `pooled` 外，每个 $(b, \text{条目}, \text{通道})$ 最多存 2 个数。保存在 `saved` 里的显存张量，写出去和读回来同样计入流量。

13 稀疏注意力前向：在线 softmax

`sparse_attn(q, kv, attn_sink, topk_idxs, softmax_scale)` 返回 `(out, saved)`，输出 `out` 与第 9 节完全一致。在架构图 8 中，这一步把“按下标抓取 KV 条目（Select top-k）”和“多查询注意力加权（Attention MQA）”融进了同一个 kernel：KV 行从显存抓到片上后直接算点积并累加进输出，不在显存里落地。

为什么要分块？一行最多有 $K = 512$ 个槽位，当 $D = 64$ 时，光把这一行的 KV 条目一次性读进片上就要 $512 \times 64 \times 4 = 128$ KiB，远超 48 KiB 容量。因此我们按 FlashAttention 的思路做**在线 softmax**：每个 (b, t) 启动一个 block，先把该位置的 $\mathtt{q}_{b,t} \in \mathbb{R}^{H \times D}$ 和 $\mathtt{attn_sink} \in \mathbb{R}^{H}$ 读上片，再把 K 个槽位切成每段 τ 个依次流式扫过。在片上，每个头 h 只需要维护三个滚动状态（代码里可命名为 `running_max`、`total`、`acc`）：

$$M_h = \mathtt{attn_sink}_h,$$
$$\ell_h = 1,$$
$$u_h = 0 \in \mathbb{R}^{D}.$$

每扫到一段槽位 $\mathcal{T}$，先用 `load` 读入这 τ 个下标 `ids`，再调 `rows, valid = sim.gather(kv, ids, (b,))` 把对应行抓到片上，算出缩放点积 $s_{h,r}$（无效槽位处设为 $-\infty$），随后用这段的新最大值 M'_h 重新缩放历史累加量：

$$M'_h = \max\Big(M_h,\ \max_{r \in \mathcal{T}} s_{h,r}\Big),$$
$$\rho_h = \exp(M_h - M'_h),$$
$$e_{h,r} = \exp(s_{h,r} - M'_h),$$
$$\ell'_h = \rho_h\, \ell_h + \sum_{r \in \mathcal{T}} e_{h,r},$$
$$u'_h = \rho_h\, u_h + \sum_{r \in \mathcal{T}} e_{h,r}\, a_r,$$

更新 $(M,\ell,u) \leftarrow (M',\ell',u')$。等全部槽位段扫完，一步归一化就能拿到最终输出和反向所需的对数分母 `lse`（log-sum-exp）：

$$\begin{aligned}\texttt{out}_{b,t,h} &= u_h/\ell_h,\\ \texttt{lse}_{b,t,h} &= M_h + \log \ell_h = \log\Big(e^{\texttt{attn_sink}_h} + \sum_r e^{s_{h,r}}\Big).\end{aligned}$$

为什么这样算一定对？不难验证，每处理完一段槽位，ℓ_h 和 u_h 始终满足不变量：

$$\begin{aligned}\ell_h &= e^{\texttt{attn_sink}_h - M_h} + \sum_{\text{已处理的 } r} e^{s_{h,r}-M_h},\\ u_h &= \sum_{\text{已处理的 } r} e^{s_{h,r}-M_h}\, a_r.\end{aligned}$$

每当遇到更大的 logit 使基准从 M_h 抬升到 M'_h 时，把旧的 ℓ_h 和 u_h 同乘修正因子 $\rho_h = \exp(M_h - M'_h)$，就等于把历史所有项的指数基准统一换成了 M'_h。而且直接拿 $\texttt{attn_sink}_h$ 初始化 M_h、拿 $1 = \exp(\texttt{attn_sink}_h - M_h)$ 初始化 ℓ_h，不仅顺手把 sink 项算进了分母，还保证 M_h 从头到尾都是有限实数——哪怕整行槽位全是 -1，跑完也是 $\ell_h = 1, u_h = 0$，输出为零向量，绝不会踩进 $-\infty - (-\infty) = \text{NaN}$ 的坑。

14 稀疏注意力反向：重算而不是保存

`sparse_attn_backward(dout, saved)` 返回 `(dq, dkv, dsink)`，数学结果与第 9 节一致。第一阶段可以偷懒把 $[B,S,H,K+1]$ 的注意力概率整张存进 cache，但在第二阶段里，这相当于每个 (b,t,h) 存 $K+1$ 个数，远超“每个 (b,t,h) 最多 4 个数”的保存上限。好在前向已经算出了对数分母 $\texttt{lse}_{b,t,h}$（每个 (b,t,h) 恰好只占 1 个数）：反向遍历每段槽位时，只要重新算一次点积 $s_{h,r}$，就能用 $\exp(s_{h,r} - \texttt{lse}_{b,t,h})$ 当场恢复概率 $p_{h,r}$。

规格与提示

- 想一想：前向结束后，每个 (b,t,h) 只需要保存哪一个标量，就够在反向看到 $s_{h,r}$ 时直接算出 $p_{h,r}$？这个标量正好满足保存上限。
- 第 9 节的反向式子完全照搬，只是改成每次处理 τ 个槽位：每段重新 gather 条目、在片上重算这段的 $p_{h,r}$ 和 $\mathrm{d}s_{h,r}$，把对 `q` 的梯度累加在片上，把对 `kv` 的梯度散射写回显存。
- 不同 query 的 block 会同时往相同的 `dkv` 行和全局的 `dsink` 上加梯度，因此写回 `dkv` 必须用 `scatter_add`，写回 `dsink` 必须用 `atomic_add`。
- 反向循环里同时存活的片上数组比前向多，分块大小 τ 通常要比前向取得更小才不会爆 48 KiB。

15 门控压缩池化：中间结果留在片上

`gated_pool(kv, score, ape)` 对应 Compressor 里“加 `ape` → 跨块重叠排布 → 逐通道 softmax 加权求和”这一整段（图 5 中从 APE、kv_state / score_state 到 sum 的部分，排布见图 6）：输入是两

个线性层投出的 `kv`, `score` $\in \mathbb{R}^{B\times S\times 2D_c}$ 以及块内位置偏置 `ape` $\in \mathbb{R}^{4\times 2D_c}$，输出是池化后的 `pooled` $\in \mathbb{R}^{B\times N\times D_c}$（尚未做 `norm` 和 RoPE）以及留给反向的 `saved`。对应的 `gated_pool_backward(dpooled, saved)` 则返回 `(dkv, dscore, dape)`，凑不满整块的尾部 token 梯度为 0。

为什么把这一段单独融成一个 kernel？如果在显存里显式拼出 $[B, N, 2m, D_c]$ 的 `values`、`logits` 和权重 `alpha`，数据量会膨胀到原来的 $2m = 8$ 倍；融进一个 kernel 后，拼接出的 $[2m, D_c]$ 块和 `alpha` 全程只待在片上，显存里只进出 $[B, S, 2D_c]$ 的输入和 $[B, N, D_c]$ 的 `pooled`。

15.1 前向

最自然的分法是每个压缩条目 (b, j) 启动一个 block，直接去显存里切它要用的那两半通道：

$$\begin{aligned}&\mathrm{load}\big(\texttt{kv},\ (b,\ jm{:}(j+1)m,\ D_c{:}2D_c)\big),\\&\mathrm{load}\big(\texttt{kv},\ (b,\ (j-1)m{:}jm,\ 0{:}D_c)\big)\qquad (j\geq 1),\end{aligned}$$

对 `score` 和 `ape` 也按完全相同的切片读入对应两半并相加，在片上拼成 $2m \times D_c$（当 $j = 0$ 时无需读前一块，直接在片上对 $m \times D_c$ 算即可）。接着按第 7 节的式子在片上算出 `alpha` 和加权和，最后只把 $[D_c]$ 的结果 `store` 进 `pooled[b, j]`。这样每个参与压缩的（token，通道）只从显存读一次，中间权重 `alpha` 根本不用写回显存。

15.2 反向

规格与提示

第 7 节的池化反向公式原封不动适用；既然一个条目的输入只有两小块，反向在片上直接重算 `alpha` 比保存下来再读回更省事。写回显存前想清楚两件事：不同条目 (b, j) 对应的 `dkv`、`dscore` 位置会不会撞车（该用 `store` 还是 `atomic_add`）？哪一个参数的梯度是所有条目共用的？

16 Indexer：流式 top-k

`index_topk(q, keys, weights, topk)` 把 Indexer 里“多头点积 → ReLU → 乘权重跨头求和 → 因果掩码 → 选 top-k”这一整串操作融进一个 kernel（图 12）：输入是已经做完 RoPE 和 Hadamard 的 index query `q` $\in \mathbb{R}^{B\times S\times H_I\times D_I}$、Indexer 的 Compressor 产出的 `keys` $\in \mathbb{R}^{B\times N\times D_I}$，以及已经乘好缩放系数的每头权重 `weights` $\in \mathbb{R}^{B\times S\times H_I}$；返回不加 offset 的整数块号张量 $[B, S, \min(\texttt{topk}, N)]$（不可读的空槽位填 -1）。

在评分的最大规模 $S = 16384, N = 4096$ 下，完整打分矩阵有六千七百多万个数，写回显存再读出来取 top-k 极不划算。流式 top-k 的做法是：每个 block 负责同一 batch 内一段连续的 query $t_0 \leq t < t_1$（共 Q 个 query），在片上维护这 Q 个 query 当前见过的最佳 $K_c = \min(\texttt{topk}, N)$ 个候选，两个数组的形状都是 $[Q, K_c]$，初值为：

$$\texttt{best_scores} = -\infty,$$

$$\texttt{best_ids} = N.$$

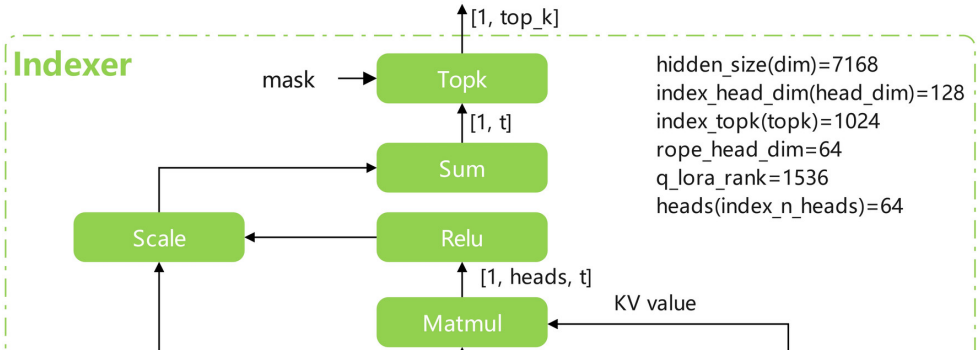


图 12 `index_topk` 融合的计算子图（截自图 7）：Matmul → Relu → Scale → Sum → mask → Topk。高达 $B \times S \times N$ 的打分矩阵全程只以分块形式在片上流过，最终落回显存的只有每个 query 选出的 `top_k` 个块号。

因为这段里最靠后的 query 是 $t_1 - 1$，编号 $j \geq \lfloor t_1/m \rfloor$ 的压缩块对这 Q 个 query 全都不可读，连读都不用读。对前面 $0 \leq j < \lfloor t_1/m \rfloor$ 的可读范围，每次从显存读入一小段共 κ 个 key（块号记为 `ids`），在片上算出 $Q \times \kappa$ 的局部打分并施加因果掩码，再用 `sim.merge_topk` 与当前最佳候选归并：

$$\texttt{part}_{t,j} = \sum_h \texttt{weights}_{b,t,h} \max\big(\texttt{q}_{b,t,h} \cdot \texttt{keys}_{b,j},\, 0\big),$$

$$\texttt{part}_{t,j} \leftarrow -\infty \qquad \text{若 } j \geq \texttt{ready}_t,$$

$$(\texttt{best_scores}, \texttt{best_ids}) \leftarrow \mathrm{merge_topk}\big(\texttt{best_scores}, \texttt{best_ids}, \texttt{part}, \texttt{ids}, K_c\big).$$

`merge_topk` 内部同样遵循“分数高者优先、同分编号小者优先”。由于全行前 K_c 名必然包含在各分段前 K_c 名的并集里，分段流式归并出来的结果与第一阶段整行调一次 `topk_lower_index_first` 完全一致。扫完所有可读 key 段后，把分数仍为 $-\infty$ 的槽位写成 -1，`store` 回显存即可。

为什么每个 block 要同时处理 Q 个 query，而不是 1 个？关键在于复用读上片的 `keys`。如果 $Q = 1$，每个 query 都要把历史上的 `keys` 自己从显存读一遍；改成 Q 个连续 query 共享一个 block 后，每段 κ 个 key 读上片一次就能同时给这 Q 个 query 打分。按因果掩码平均读一半历史块估算，读 `keys` 的总显存流量大约是：

$$T_{\text{keys}} \approx \frac{S}{Q} \cdot \frac{N}{2} \cdot D_I \cdot 4 \text{ 字节}.$$

显然 Q 越大，读 key 的流量就按 $1/Q$ 成倍下降；但与此同时，片上要同时塞下 Q 个 query、κ 个 key、$[Q, \kappa]$ 的局部打分以及 `merge_topk` 拼接出的 $[Q, K_c + \kappa]$ 临时数组，所以 Q 和 κ 不能无限放大，需要卡着 48 KiB 容量选一对平衡点。

17 整层前向：把融合 kernel 接起来

最后一步，`attention_forward(x, params, args)` 把整层前向搬到模拟器上跑通（对应图 2 的整个橙框）。其中 x 是 $[B, S, E]$ 的显存张量，`args` 是配置对象 `ModelArgs`，字典 `params` 则把 `model.named_parameters()` 里的每个参数名（如 `"wq_a.weight"`、`"compressor.ape"` 等）映射到对应的显存张量，并额外提供预拆好的 `"rope.cos"` 与 `"rope.sin"`（形状均为 $[\texttt{max_seq_len}, d_r/2]$，即 `freqs_cis` 的实部和虚部）。

整层里的常规稠密算子无需重复造轮子，直接调框架 csa/kernels.py 提供的四个宿主函数即可。它们内部都已经按 48 KiB 做好了分块，产生的显存流量同样计入整层总账：

框架宿主函数	功能说明
`linear(x, weight, groups=1, scale=1.0)`	分块计算 $\texttt{scale}\cdot x W^\top$；当 groups >1 时将 x 最后一维均分成 groups 组，分别与 weight 对应行块相乘（专用于分组投影 wo_a）
`map_rows(fn, x, *tensors)`	将 x 按行分块读上片，对每块调用你传入的 fn(rows, *tensors) 再写回；tensors 是每块都需要完整读入的小张量（如 RMSNorm 的 weight）
`rope(x, cos, sin, rd, step=1, inverse=False)`	对 x[:, n] 在位置 $n\cdot$step 上做 partial RoPE；普通 token 用默认 step=1，压缩条目用 step=4
`hadamard(x)`	沿最后一维做归一化 Hadamard 变换

注意框架没有单独提供 RMS 归一化的 kernel，而是让你通过 map_rows 在片上复用第一阶段写好的 rms_forward：比如 attention query 的每头无权重归一化就是 kernels.map_rows(lambda rows: rms_forward(rows, args.norm_eps)[0], q)；带权重的 RMSNorm 则把对应的 weight 张量作为额外参数传给 map_rows，在 lambda 里乘上去即可（传入 fn 的已经是片上 Local 数组，直接用 NumPy 算符运算）。

除了框架提供的这四个函数和你前面实现的三个融合算子（gated_pool、index_topk、sparse_attn）外，你还需要在 fused.py 里补两个很简单的小 kernel：一个用来生成滑动窗口槽位并与 index_topk 的输出（有效块号加偏移 S，-1 不变）拼成完整的 topk_idxs；另一个用来把局部 KV local 和 Attention 的 Compressor 输出的 compressed 写进同一个 $[B, S+N, D]$ 的 kv 张量。把这些积木按第 10 节的顺序串起来，整层前向就跑通了：

1. **Attention query：** 用 linear 和带权重的 map_rows(rms_forward) 算出共享低秩表示 qr；再经 linear、reshape 成 $[B, S, H, D]$、无权重的 map_rows(rms_forward) 和 rope 得到 q。
2. **局部 KV：** 对 x 依次调 linear、带权重的 map_rows(rms_forward) 和 rope，得到 local。
3. **两个 Compressor：** Attention 的和 Indexer 的 Compressor 各自先调两次 linear 算出 kv 与 score 投影，送入你的 gated_pool，再过带权重的 map_rows(rms_forward) 和 step=4 的 rope；Indexer 的 Compressor 末尾多调一次 hadamard。
4. **Indexer 选块：** qr 经 linear、reshape、rope、hadamard 得到 index query；x 经带 scale $= D_I^{-1/2} H_I^{-1/2}$ 的 linear 得到每头权重；连同 Indexer 的 Compressor 输出的 keys 一起送入你的 index_topk。
5. **拼接与稀疏注意力：** 启动你写的两个小 kernel 造好 topk_idxs 和 kv，调你的 sparse_attn 拿到 o。
6. **输出投影：** o 经 inverse=True 的 rope，reshape 成 $[B, S, HD]$，先后调 groups=G 的 linear（wo_a）和普通 linear（wo_b）得到 y。

评分器会同时加载你同一目录下的 `attention.py` 和 `fused.py`，先核对输出 y 与第一阶段参考解一致，再检查整次 `attention_forward` 的累计流量不超过参考解的 1.5 倍加 4 KiB。参考解就是老老实实按上面 6 步调下来的，没有做跨算子的额外融合，因此只要前面的融合算子写对，流量预算非常宽裕。如果你有兴趣进一步压低流量，也可以尝试把相邻的逐行算子（如 RMS 和 RoPE）合进同一个自定义 kernel，或者让局部 KV 的 RoPE 直接写进 `kv` 的前 S 行以省掉一次搬运。

18 流量估算

写完 kernel 后，怎么判断自己的分块方案有没有冤枉流量？其实只需抓大头。以稀疏注意力前向为例，单个位置 (b, t) 读写的显存元素数（乘 4 即字节数）为：

$$\underbrace{HD}_{\text{读q}} + \underbrace{H}_{\text{读 sink}} + \underbrace{K}_{\text{读槽位}} + \underbrace{K_{\text{有效}}\, D}_{\text{gather KV 条目}} + \underbrace{HD + H}_{\text{写 out 和 lse}}.$$

当 $K_{\text{有效}} \approx 410, D = 64, H = 4$ 时，中间那项 gather KV 条目占了约 2.6×10^4 个元素，而读写 `q`、`out` 加起来才五百多个元素。看清主项之后，不同写法的账一算便知：

- 如果不写在线 softmax，而是拆成 “第一个 kernel 算完所有 logits 写回显存、第二个 kernel 读出 logits 做 softmax 再 gather 一遍 KV 求和”，那么主项 $K_{\text{有效}}\, D$ 被硬生生读了两遍，总流量直接翻倍，必然超出 1.5 倍的预算线。
- 如果前向把 $H(K+1)$ 个注意力概率写回显存留给反向，虽然相比庞大的 $K_{\text{有效}}\, D$ 流量只多出几个百分点，但保存的元素个数是上限（$4H$）的一百多倍，会被保存量检查直接拦下。

另外两个融合算子也是同样的道理：门控池化的主项是把每个参与压缩的（token，通道）上的 `kv` 和 `score` 各读一次（反向各读一次再写一次）；流式 top-k 的主项是第 16 节算过的 T_{keys}，只要按 Q 个 query 分块复用了 key，流量就能压下来。

A 实现核对清单

提交前可以对照下面 10 条逐一打勾，绝大部分常见丢分点都在这里：

1. **形状约定：** `Linear` 的权重形状是 $[\text{out}, \text{in}]$，前向算 $x W^\top$；共享 `kv` 表没有头维度；`topk_idxs` 最后一维宽度为 $\min(S, w) + \min(k, N)$。
2. **RMS 归一化：** 全零行输入必须输出全 0 而非 NaN；`RMSNorm` 的权重梯度要沿所有前导维度求和。
3. **Compressor：** 逐通道沿 $2m$ 个位置做 softmax；前 D_c 个通道留给下一块、后 D_c 个通道归当前块；第 0 个条目的前 m 个门控 logit 填 $-\infty$；凑不满整块的尾部 token 梯度为 0；RoPE 位置取块起点 mj（步长 m）；仅在 `self.rotate=True` 时做 Hadamard；当 $S < m$ 时 `forward` 返回 `None`。

4. **Indexer：**每个头先点积再过 ReLU，乘每头权重后跨头求和；每头权重只乘 $D_I^{-1/2}H_I^{-1/2}$，不做 softmax；在取 top-k **之前**把 $j \geq \texttt{ready}_t$ 的分数屏蔽为 $-\infty$；用 `topk_lower_index_first` 选块；选出的块号若 $\geq \texttt{ready}_t$ 改写成 -1，有效块号加上 `offset`；`backward_scores` 里被屏蔽位置的梯度为 0。
5. **稀疏注意力：**局部槽位、压缩槽位和每头的 $\texttt{attn_sink}_h$ 放在同一个分母里归一化；取指数前先减去含 sink 在内的最大值；槽位为 -1 的项不参与；重复的有效槽位各自独立算一次；整行无有效槽位时输出全 0。
6. **整层串联：**滑动窗口包含当前位置 t；`kv` 表前 S 行放 `local`、后 N 行放 Attention 的 Compressor 输出；注意力输出 o 先做**逆** RoPE 再做分组投影 `wo_a`；不要忘了把路由表存进 `self.last_topk_idxs`。
7. **手写反向：**所有参数梯度用 += 累加进 `_grads`；`dkv` 写回时用 `np.add.at` 汇集所有头、所有 query 及重复槽位的梯度；正向 RoPE 的转置是逆 RoPE，逆 RoPE 的转置是正向 RoPE，Hadamard 的转置是它自身；整数路由表不传梯度；`Attention.backward` 收到 `dindex_scores` 时，要把 `backward_scores` 返回的两项梯度分别加进 $\mathrm{d}x$ 和 `dqr`。
8. **类型与纯度：**`float32` 输入必须返回 `float32` 输出；严禁原地修改调用方传入的数组。
9. **第二阶段内核：**宿主代码绝不碰张量数值；分块大小兼顾 48 KiB 容量与 1.5× 流量线；多 block 并发写同一位置时必须用 `atomic_add` 或 `scatter_add`；`saved` 保存的额外元素数不超过 docstring 上限。
10. **模拟器整层前向：**attention query 用 `map_rows` 调无权重 `rms_forward`；压缩条目做 RoPE 时 `kernels.rope` 要传 `step=4`；`index_topk` 返回的有效块号要加偏移 S（-1 保持不变）；Indexer 的每头权重用 `kernels.linear` 算，`scale` 取 $D_I^{-1/2}H_I^{-1/2}$。

B 用有限差分检查手写反向

手写反向最常见的错误是漏掉一条梯度路径、写反一个转置或者少乘一个常数。这类错误不会报错，只会悄悄算出错误的数。有限差分检验不看你的反向公式，而是直接用前向“试”出梯度，再和反向的结果逐元素对比。

B.1 原理

设 L 是一个标量，x_i 是输入或参数中的某一个元素。把 x_i 分别加上和减去一个很小的数 h，各跑一次前向，就能近似出 L 对它的偏导：

$$\frac{\partial L}{\partial x_i} \approx \frac{L(x + h\,e_i) - L(x - h\,e_i)}{2h},$$

其中 e_i 是只在第 i 个位置为 1 的单位向量。对每个元素都做一遍，就得到一张与 x 同形状的数值梯度，再与反向算出的解析梯度逐元素比较。这种两边各扰动一次的写法叫中心差分，截断误差是 $O(h^2)$；只往一边扰动的 $\big(L(x+h\,e_i)-L(x)\big)/h$ 误差是 $O(h)$，精度差得多。h 也不能太小：$L(x+h\,e_i)$ 和 $L(x-h\,e_i)$ 非常接近，相减会丢掉有效数字，h 越小舍入误差越大。在 float64 下取 $h = 10^{-6}$，两种误差都足够小。

B.2 把反向变成可检查的标量

反向函数接收的是上游梯度 $\mathrm{d}y$，而不是一个标量损失。只要把检查用的目标取成

$$L = \sum y \odot \mathrm{d}y,$$

就有 $\partial L/\partial y = \mathrm{d}y$。于是 `backward(dy)` 返回的 $\mathrm{d}x$ 应当等于 L 对 x 的数值梯度，累加进 `_grads` 的每个参数梯度也应当等于 L 对该参数的数值梯度。$\mathrm{d}y$ 用随机数即可，这样每个输出元素都会参与检查。

B.3 用 `gradcheck.py` 检查

`numerical_gradient(objective, array, step=1e-6)` 对 `array` 的每个元素做上面的中心差分：它就地改写 `array` 的一个元素，调用无参函数 `objective()` 得到标量，再把元素恢复原值。因此 `objective` 必须读取同一个数组对象，例如直接用 `layer.weight`，而不是它的拷贝。下面的例子改写自 `test_public.py` 中检查 `Linear` 反向的测试：

```
import numpy as np
from csa import attention as impl
from csa.gradcheck import numerical_gradient

rng = np.random.default_rng(31)
layer = impl.Linear(5, 3, np.random.default_rng(32))
x, dy = rng.normal(size=(2, 4, 5)), rng.normal(size=(2, 4, 3))

layer(x)                    # 前向把 x 存进 layer._x，留给反向
layer.zero_grad()
dx = layer.backward(dy)
dweight = dict(layer.named_gradients())["weight"].copy()

def objective():            # L = sum(y * dy)，于是 dL/dy = dy
    return np.sum(layer(x) * dy)

num_dx = numerical_gradient(objective, x)
num_dw = numerical_gradient(objective, layer.weight)
np.testing.assert_allclose(dx, num_dx, atol=1e-8, rtol=1e-6)
np.testing.assert_allclose(dweight, num_dw, atol=1e-8, rtol=1e-6)
```

检查整层时用 `attention_gradients(model, x, dy)`：它返回一个字典，把输入 `"X"` 和每个参数名映射到一对数组 `(analytic, numerical)`，并且已经替你固定好了路由（见下面的注意事项）。公开测试都用 `np.testing.assert_allclose` 比较，相对误差容差在 10^{-6} 到 3×10^{-5} 之间。

B.4 注意事项

- **只在很小的配置上做**。每个元素都要跑两次完整前向，总开销与元素个数成正比。所以公开测试只用 $E = 3$、2 个头、序列长度不超过 15 的配置；评分用例规模大，改为直接与参考实现比对。
- **目标在扰动范围内必须光滑**。top-k 是离散选择，输入只抖动 h 也可能换掉一个入选的块，让 L 跳变，差分结果就失去意义。`csa.ops` 里的 `record_routes` 会记下每次 `topk_lower_index_first`

的结果，`replay_routes` 让之后的调用按顺序原样返回这些结果；`attention_gradients` 就是先记录一次、再在每次扰动时回放。这也是所有 top-k 都必须走 `topk_lower_index_first` 的原因。ReLU 在 0 点不可导也是同类问题，不过随机输入下几乎碰不到。

- **一律用 float64**。float32 只有约 7 位有效数字，$h = 10^{-6}$ 时 $L(x + h\,e_i)$ 与 $L(x - h\,e_i)$ 的差会被舍入误差淹没。
- **从小模块往上查**。先单独检查 `Linear`、`rms_backward` 这些小模块，再查 `sparse_attn_backward` 和 `Compressor.backward`，都通过后再检查整层。整层对不上时，先看是哪个参数的梯度出错，问题通常就在那条支路上；再看出错的元素集中在哪里（只在序列开头几个位置、只在尾部 token、只在 RoPE 通道），往往能直接定位到某个边界条件。

C 作业之外的真实模型细节

为了让作业聚焦在 CSA 的核心算法与存储分块上，框架省去了生产级推理代码里的几处外围工程细节，这里做个简要交代：

C.1 RoPE 频率与 YaRN 长上下文扩展

`csa/rope.py` 中的 `csa_freqs(args)` 调用 `precompute_freqs_cis` 生成旋转频率，基础底数 θ 取 `compress_rope_theta`。对通道对 $i = 0, \ldots, d_r/2 - 1$：

$$\omega_i = \theta^{-2i/d_r},$$
$$\texttt{freqs_cis}_{t,i} = e^{\mathrm{i}\,t\,\omega_i}.$$

当配置中的原始训练长度 $L_0 = \texttt{original_seq_len} > 0$ 时，启用 YaRN 分频段插值。记扩展倍数为 $F = \texttt{rope_factor}$，高低频分界下标由波长比 $\beta_{\text{fast}}, \beta_{\text{slow}}$ 决定：

$$c(\beta) = \frac{d_r \log\big(L_0/(2\pi\beta)\big)}{2\log\theta},$$
$$a = \max\big(\lfloor c(\beta_{\text{fast}})\rfloor, 0\big),$$
$$b = \min\big(\lceil c(\beta_{\text{slow}})\rceil, d_r - 1\big),$$
$$\lambda_i = 1 - \mathrm{clip}\Big(\frac{i-a}{b-a}, 0, 1\Big),$$
$$\omega_i' = \frac{\omega_i}{F}(1-\lambda_i) + \omega_i\lambda_i.$$

若 $a = b$ 则给 b 加上 0.001 防止除零。默认测试配置下 `original_seq_len=0`，直接使用未插值的 ω_i。

C.2 逐 token 的 Decode 模式

官方源码同时支持 prefill 和逐 token 生成的 decode，图 1 就是按 decode 阶段画的：在 decode 时，局部窗口 KV cache 是一个宽度固定为 w 的环形缓冲区；Compressor 则用 `kv_state` 和 `score_state` 缓存当前尚未凑满 m 个 token 的半块投影（即图 6 中的 cur_state），每攒够 m 个 token 就触发一次池化，往压缩缓存里追加一行；此时 `kv` 表前 w 行是窗口缓存，因此 Indexer 槽位的偏移量 `offset` 是 w 而不是 S。本作业只考察 prefill。

C.3 FP8 / FP4 低比特量化

官方推理代码在局部 KV 和 Attention 的压缩条目进入缓存前调用 `act_quant` 模拟 FP8 量化，在 Indexer 的 query 和 key 点积前调用 `fp4_act_quant` 模拟 FP4 量化（这正是 Indexer 要做 Hadamard 旋转的原因），线性层则根据权重类型分发到 BF16、FP8 或 FP4 GEMM。本作业将这两个量化钩子设为了恒等函数。

C.4 多卡张量并行（TP）

在多 GPU 张量并行下，Attention 的 H 个头、Indexer 的 H_I 个头以及输出投影的 G 个组均按设备切分；Indexer 在取 top-k 前需要先对各卡算出的局部打分做一次 `all_reduce`，`wo_b` 的输出同样需要 `all_reduce`。本作业对应单卡情形（`world_size=1`）。

C.5 全景图里的其余模块

图 1 左侧的 HCA（Heavily Compressed Attention）是与 CSA 交替堆叠的另一种注意力层：它采用 128 : 1 的无重叠大步长压缩（`coff * compress_ratio = 1 * 128`），不设 Indexer，每个 query 直接读取全部历史压缩块。图中央的 `hc_pre` / `hc_post` 是多流残差超连接（mHC），上方的 Hash 层与 MoE 则是前馈网络部分。

D 与官方源码的对照

随作业发放的 `reference/deepseek_v4/` 是官方推理代码的原样副本，固定在下面这个版本；同目录的 `SOURCE.md` 还记录了每个文件的 SHA-256。

- 仓库：huggingface.co/deepseek-ai/DeepSeek-V4-Pro
- commit：b5968e9190ef611bbf34a7229255be88a0e937c1
- 文件：仓库 `inference/` 目录下的 `model.py` 和 `kernel.py`，与本地副本同名

作业框架沿用了官方的类名、属性名和张量排布。下表按名称对应，在源码里搜索这些名字即可找到：

讲义章节	文件	官方源码中的名称
RMSNorm（第 5 节）	`model.py`	`RMSNorm`
RoPE（第 6 节）	`model.py`	`precompute_freqs_cis`、`apply_rotary_emb`
Hadamard 旋转（第 6 节）	`model.py`	`rotate_activation`
Compressor（第 7 节）	`model.py`	`Compressor`；跨块重叠排布在它的 `overlap_transform` 方法里
Indexer（第 8 节）	`model.py`	`Indexer.forward`
滑动窗口槽位（第 10 节）	`model.py`	`get_window_topk_idxs`

讲义章节	文件	官方源码中的名称
整层串联（第 10 节）	`model.py`	`Attention.forward`：先算 query 和局部 KV，再调用 `self.indexer`、`self.compressor` 和 `sparse_attn`，最后做逆 RoPE 和 `wo_a`、`wo_b` 投影
稀疏注意力与在线 softmax（第 9、13 节）	`kernel.py`	`sparse_attn` 及其调用的 TileLang kernel `sparse_attn_kernel`

官方仓库只有推理前向；本作业的反向推导、两级存储模拟器、重算和流式 top-k 都是课程自己的设计。CSA 的算法背景可参阅 DeepSeek-V4 技术报告第 2.3 节。